

DeepSafe: A Cryptography Random Verification Layer

DeepSafe Team

Abstract—DeepSafe is an open, permissionless verification layer that decentralizes trust by distributing verification authority across a globally dispersed network of independent nodes. Each request—whether for cross-chain asset transfers, oracle data feeds, asset custody operations, or AI data attestations—is handled by a randomly selected, identity-hidden committee chosen via Ring Verifiable Random Function (Ring VRF). Verifiers run inside Trusted Execution Environments (TEE) to ensure hardware-enforced confidentiality, generate non-interactive zero-knowledge proofs to preserve auditability without revealing sensitive data, and collaborate through threshold Multi-Party Computation (MPC) to produce collusion-resistant signatures. DeepSafe integrates seamlessly with existing blockchains, delivering millisecond-level verification, horizontal scalability, and strong resistance to censorship or coordinated attacks. Anchored by a staking-and-slashing economic model and guided by community-driven governance, DeepSafe aligns participant incentives and provides transparent cryptographic guarantees, forming the foundational infrastructure for truly trustless collaboration and frictionless asset mobility in the Web3 era.

Index Terms—evolving committee, threshold cryptography, ring VRF, trusted execution environment

I. BACKGROUND

As the Web3 ecosystem rapidly evolves, security, verifiability, and decentralization remain the most critical and challenging imperatives. While decentralized consensus mechanisms have successfully addressed the trust problem at the “ledger” level, broader modules such as data interactions, asset custody, oracle input, cross-chain communication, and AI-driven computation continue to heavily rely on centralized service providers. These “trust intermediaries” fundamentally contradict the core principles of Web3—trustlessness and censorship resistance—while introducing significant security risks and systemic vulnerabilities.

Against this backdrop, DeepSafe emerges as an independent, open, and transparent decentralized verification node network, envisioned as a trusted bridge between on-chain consensus and off-chain information. The core mission of DeepSafe is to liberate verification authority from centralized entities and redistribute it across a globally distributed, permissionless node network, enabling automated, collusion-resistant, and cryptographically auditable verification of transactions, states, cross-chain messages, and off-chain data.

At present, several core components of the Web3 stack exhibit critical centralized dependencies:

- Asset custody typically relies on centralized custodians or smart contract administrators, resulting in users relinquishing full control over their assets.
- Cross-chain communication heavily depends on a limited set of nodes or committees for signature validation, introducing considerable attack surfaces and collusion risks.
- Oracle systems are controlled by a few centralized data sources, lacking robust verification mechanisms and making on-chain contracts vulnerable to manipulation or erroneous execution.
- Bitcoin and other core crypto assets have yet to achieve truly decentralized liquidity release and on-chain mapping due to centralized custody and verification bottlenecks.
- As AI further integrates with blockchain, emerging use cases such as data attestation, model invocation, and training verification demand a neutral and trustworthy verification foundation.

Recent incidents, such as the collapse of FTX, the \$1.5 billion Bybit wallet hack, WBTC reserve controversies, and Multichain’s centralized private key vulnerabilities, all underscore the inherent risks of centralized verification architectures. Even though blockchains provide immutable ledgers, this “record-only” security model fails to safeguard off-chain assets, cross-chain transfers, and AI data processes.

DeepSafe does not aim to build a new public chain nor to replace existing consensus layers; rather, it serves as an independent, universal “decentralized verification layer.” By integrating cutting-edge cryptographic and hardware security techniques—such as Ring VRF (Verifiable Random Function), Zero-Knowledge Proofs (ZKP), Trusted Execution Environments (TEE), and Multi-Party Computation (MPC)—DeepSafe establishes an efficient, collusion-resistant, auditable, and privacy-preserving verification network.

Any individual, institution, AI agent, or DApp can invoke the DeepSafe network to obtain automated, transparent, and cryptographically auditable verification results without relying on any centralized intermediary. If blockchain technology has enabled “decentralized bookkeeping” on a global scale, DeepSafe further advances this paradigm by realizing “decentralized verification” as the foundational trust architecture. Through this framework, DeepSafe becomes the essential infrastructure for asset custody, cross-chain bridges, oracles, AI data attestation and auditing, Bitcoin mapping, and cross-chain asset management. It ensures that every cross-chain transfer, asset collateralization, high-value transaction, and AI inference can be verified securely and transparently without centralized trust.

The “verification over trust” paradigm introduced by DeepSafe will drive the trustworthy evolution of decentralized finance, AI networks, and the global data economy, becoming a fundamental security cornerstone for future digital collaboration and asset free flow.

II. TECHNICAL OVERVIEW

A. Introduction

DeepSafe is an open, decentralized, and secure Cryptographic Randomized Verification Layer designed to serve as a trust backbone for Web3, AI, and various cross-chain or data collaboration scenarios. By leveraging a permissionless and globally distributed network of nodes, DeepSafe provides a universal, cryptographically provable verification service that completely eliminates dependence on centralized intermediaries.

The core philosophy of DeepSafe is to fundamentally liberate “verification authority” from a small set of centralized institutions, nodes, or individuals, redistributing it to a distributed network composed of global participants. Within the DeepSafe architecture, each cross-chain asset transfer, state signature, asset unlocking, or data invocation is executed by a small, randomly selected committee of verifiers. These verifiers are chosen via cryptographic random sampling mechanisms (e.g., Ring VRF), making them identityless, unpredictable, and uncolludable, while preventing any single verifier from independently controlling assets or operations — effectively mitigating potential malicious behaviors.

Compared to traditional centralized notarization solutions, DeepSafe preserves excellent user experience and cross-chain compatibility, while leveraging advanced cryptographic and hardware security techniques (e.g., Zero-Knowledge Proofs, Multi-Party Computation, Trusted Execution Environments) to achieve full decentralization and censorship resistance at the verification layer. The verifier network can be viewed as a “hidden committee” dynamically generated through cryptographic randomness, where all verification tasks are executed anonymously within secure, isolated environments. This process remains fully transparent and publicly auditable while being resistant to interference.

Any participant can freely deploy and operate a DeepSafe node, engage in the verification process, and earn economic incentives, transforming verification into a public, trustless, and incentivized activity akin to “mining,” thereby enhancing overall network security and autonomy.

For example, during the initial stage, committee members collaboratively generate addresses and accounts on each supported blockchain using Multi-Party Computation (MPC). When a user intends to transfer tokens from chain A to chain B, they first deposit the assets into a DeepSafe Network multi-signature account on chain A and submit a transfer request. Subsequently, the committee jointly signs a transaction on chain B via a Threshold Signature Scheme (TSS), issuing the corresponding tokens to the user’s address on chain B. This mechanism ensures decentralization, transparency, and security throughout the cross-chain process.

To achieve evolving and hidden committee, Benhamouda *et al.* [2] proposed a complicated two-committee scheme, where a public nominating committee is first selected by cryptographic sortition, and then each nominating committee members will randomly select a holding committee member. Alternatively, we present a much more efficient solution based on a new notion, called *Ring VRF* (R-VRF). In Ring VRF, the

real public key of a verifiable random function (VRF) instance can be hidden among a ring. More specifically, we design a non-interactive zero knowledge protocol that allow the prover to show that he/she knows the sk corresponding to one of the public keys among the ring and the output of the pseudorandom function (PRF) $PRF_{sk}(x)$ is correct. In particular, showing $PRF_{sk}(x) = v$ w.r.t. the ring $(pk_0, \dots, pk_{N-1})$, we have the following relation

$$\mathcal{R}_{VRF} = \left\{ \begin{array}{l} ((pk_0, \dots, pk_{N-1}, x, v), sk, \ell) \mid \\ \ell \in \{0, \dots, N-1\} \wedge (pk_\ell, sk) \in \mathcal{R}_{pk} \wedge \\ v \leftarrow PRF_{sk}(x) \end{array} \right\}$$

In addition, to achieve minimal malware attack interface, we deploy all the key-related programs on a trusted execution environment (TEE), such as Intel SGX, to make sure that even a malicious committee member cannot extract his/her secret shares. Moreover, TEE enforces key share erasure after committee switching.

B. Preliminaries

1) *Hybrid Encryption*: Our scheme utilizes hybrid encryption over blockchain for peer-to-peer communication. The hybrid encryption scheme HEnc consists of the following 4 PPT algorithms.

- $\sigma \leftarrow \text{Setup}(1^\lambda)$: take input as security parameter $\lambda \in \mathbb{N}$, and output a group parameter σ .
- $(pk, sk) \leftarrow \text{KeyGen}(\sigma)$: pick random $s \leftarrow \mathbb{Z}_q^*$ and set $h := g^s$, and output $(pk := (g, h), sk := s)$.
- $C \leftarrow \text{Enc}_{pk}(\sigma; m)$: pick random $r \leftarrow \mathbb{Z}_q$; compute $c_1 := g^r$ and $c_2 := h^r$; set $k \leftarrow \text{hash}(c_2)$; compute $u := \text{AES-GCM}_k(m)$; output $C = (c_1, u)$.
- $m \leftarrow \text{Dec}_{sk}(\sigma; C)$: compute $c_2 := (c_1)^{sk}$; set $k \leftarrow \text{hash}(c_2)$; output $m := \text{AES-GCM}_k^{-1}(u)$.

Clearly, the above hybrid encryption scheme HEnc is IND-CPA secure under the DDH assumption and the semantic security of AES-GCM encryption mode. In the actual implementation, hybrid encryption requires a trusted curve group parameter, and we adopt the well-known curve 25519.

2) *Commitment*: A commitment scheme allows a sender to commit to a value, and later he can reveal the value by opening the commitment. A secure commitment scheme shall have two properties: hiding and binding. We denote a commitment as $c = \text{Com}(m; r)$, where m is the committed value(message) and r is the randomness. The hiding property and binding property are defined as follows:

- *Hiding*: for any PPT adversary $\mathcal{A}$, $\Pr[(m_0, m_1) \leftarrow \mathcal{A}; b \leftarrow \{0, 1\}; c \leftarrow \text{Com}(m_b) : \mathcal{A}(c) = b] \approx \frac{1}{2}$
- *Binding*: for any PPT adversary $\mathcal{A}$, the probability $\Pr[(m_0, r_0, m_1, r_1) \leftarrow \mathcal{A} : m_0 \neq m_1, \text{Com}(m_0; r_0) = \text{Com}(m_1, r_1)] \approx 0$

A Pedersen commitment [16] is defined as follows. $c := \text{Com}_{ck}(m; r) := g^m h^r$, where $ck = h$ is the commitment key whose discrete logarithm is unknown to the committer. Pedersen commitment is additively homomorphic; namely, $\text{Com}_{ck}(m_0; r_0) \cdot \text{Com}_{ck}(m_1; r_1) = \text{Com}_{ck}(m_0 + m_1; r_0 + r_1)$.

In some use case, we don’t need homomorphic property. We simply use the salted hash based commitment defined as $c := \text{Com}(m; r) = \text{hash}(m, r)$.

3) *Blockchain*: Our system is built on top of a blockchain platform. We assume parties can use the underlying blockchain as a non-blocking broadcast channel. Namely, if party P_1 posts a message m_1 over the blockchain at the i -th round, any other parties will receive it no later than $(i + \delta)$ -round, where δ is a known bound. For recording wealth, the blockchain also serves as an account model ledger so that stake holders can lock some amount of tokens to apply for a committee member. Formally, we abstract the blockchain platform as a UC functionality $\mathcal{G}_{\text{blockchain}}$ in Fig. 3 in Sec. II-C, below.

4) *Verifiable Random Functions and Cryptographic Sortition*: A verifiable random function (VRF) [14] is a pseudorandom function that enables the key holder to prove the correctness of the output. A VRF scheme VRF consists of the following 3 PPT algorithms:

- $(pk, sk) \leftarrow \text{KeyGen}(\lambda)$: The key generation algorithm produces a public key pk and a secret key sk .
- $(v, \pi) \leftarrow \text{Eval}_{sk}(x)$: The evaluation algorithm takes secret key sk and pre-image x , and image v and a proof π .
- $0/1 \leftarrow \text{Verify}_{pk}(x, v, \pi)$: The verifier algorithm takes input as the public key pk , the pre-image x , the image v , and the proof π , and it outputs 0 for rejection or 1 for acceptance.

Besides pseudo-randomness, a secure VRF scheme should also have the following properties.

- **Completeness**: for any x , given $(v, \pi) \leftarrow \text{Eval}_{sk}(x)$, we have $\text{Verify}_{pk}(x, v, \pi) = 1$.
- **Uniqueness**: no PPT adversary $\mathcal{A}$ can output a public key pk , a pre-image x , and (v_1, π_1, v_2, π_2) such that $v_1 \neq v_2$ and $\text{Verify}_{pk}(x, v_1, \pi_1) = \text{Verify}_{pk}(x, v_2, \pi_2) = 1$.

A formal definition can be found in [14].

It is well-known that cryptographic sortition [7] can be realized by VRF schemes. Over the blockchain, suppose (pk_i, sk_i) are associated with party P_i . For a public input x , each party P_i can compute $(v_i, \pi_i) \leftarrow \text{VRF.Eval}_{sk_i}(x)$. Then, v_i can be used to select the committee with public verification. In our system, we would like to further hide the committee member's identities by proposing a new notion called Ring VRF (cf. Sec. II-D1, below).

5) *Trusted Execution Environment*: Trusted execution environment (TEE) is designed to guarantee confidentiality and integrity of computations. It is an isolated part that can store sensitive data and can issue attestation to prove correctness of computation. In practice, Intel SGX and ARM Trustzone are popular candidates of TEE. Although a few side-channel attacks, e.g. [21], [12], [20], have been explored against those TEE candidates, new designs and fixes are proposed on a monthly basis [4], [17], [13], [15], [19]. Hence, we take TEE as an acceptable hardware guard to ensure privacy of secret key components. In this work, our benchmarks are executed on the Intel SGX platform for its readily deployed remote attestation infrastructure; however, we emphasize that our protocol can also be implemented on any other TEE platforms.

C. System Overview and Security Model

1) *System Architecture*: In this section, we will give an overview of DeepSafe platform with the example of cross-chain transfer between Bitcoin (BTC) and Ethereum (ETH).

As depicted in Fig. 1, (C_1, C_2) denotes the cross-chain channel creation procedures; (A_1, A_2, A_3) denotes the lock-in procedures; (B_1, B_2, B_3) denotes the lock-out procedures.

Initially, a user creates a DeepSafe account on the Bitcoin blockchain through the DeepSafe committee (cf. C_1) and a smart contract anchored to the DeepSafe account on Ethereum (cf. C_2); subsequently, the DeepSafe committee members will sign a test message to check the channel's availability.

Each committee member runs light nodes of BTC and ETH to verify transactions. When the user wants to lock-in 1 BTC, he makes a transfer of 1 BTC to the address that was created in step C_1 (cf. A_1). After confirming the transaction, the DeepSafe committee will use threshold signature to sign a transaction that creates 1 wBTC on Ethereum (cf. A_2). Finally, the signed transaction will be submitted to Ethereum (cf. A_3). The lock-out process is similar: first, the user destroys 1 wBTC on the Ethereum Anchor contract and specify the beneficiary address (cf. B_1). Then, the committee will monitor the event and generate the corresponding Bitcoin transaction (cf. B_2). Lastly, the transaction will be broadcast on Bitcoin network (cf. B_3).

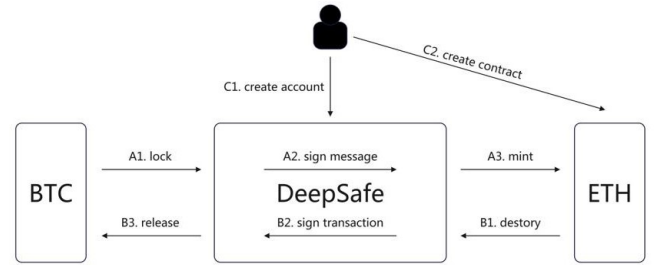


Fig. 1: DeepSafe overview

DeepSafe supports evolving hidden committee. We assume the underlying blockchain has a weak beacon oracle that can periodically produce an unpredictable but presumably biased random string. For instance, the hash digest of latest blocks. Define an epoch as a pre-defined number of rounds. At the beginning of each epoch, a new committee will be selected by cryptographic sortition using Ring VRF. The old committee will then handover the secret keys to the new committee in a secure and verifiable way. Fig. 2 shows an overview of the evolving hidden committee. More specifically, it consists of the following phases.

1. **Registration**. Before an epoch starts, all DeepSafe stakeholders can register for becoming an committee member. The registration phase works as follows. A stakeholder locks a certain amount of stake together with a public key. Later, the associated public key will be used for cryptographic sortition. For the sake of fairness and animosity, our system utilizes the flat mode, i.e., there is a fixed amount of stake to be locked per registration. If a stakeholder holds more stake, he/she can register several times to get a larger winning probability.
2. **Sortition**. For committee sortition, everyone first generates the weak beacon string x over DeepSafe blockchain, which can be the hash of the previous blocks. Then, each stakeholder takes x as input and computes $\text{PRF}_{sk}(x)$. If

- $\text{PRF}_{\text{sk}_i}(x) = v < T$, then P_i randomly selects a ring $(\text{pk}_1, \dots, \text{pk}_\ell)$ and posts message $(\text{pk}_1, \dots, \text{pk}_\ell, v, \pi, \text{epk})$ over the blockchain, where π is the Ring VRF proof, v is the PRF output, epk is a freshly generated ephemeral public key. Winning nodes are elected for next-epoch committee. From then on, they can be communicated with, using the ephemeral public key epk . As we can see, the ephemeral public key reveals nothing about the committee member's identity, and the real identity is hidden within the ring $(\text{pk}_1, \dots, \text{pk}_\ell)$.
3. **Handover.** When a new committee is elected, the shared secret keys need to be passed from the previous-epoch committee to the new committee while keeping the public keys unchanged. Our protocol is maliciously secure with identifiable aborts. See sec. II-E for details.
 4. **Threshold Signature.** All transactions in our DeepSafe shall be jointly signed by the committee using a threshold signature scheme. In particular, the threshold signature scheme for ECDSA [11] is adopted from the GG20 scheme by Gennaro and Godfeder [9] where the malicious party can be identified in a protocol abortion.

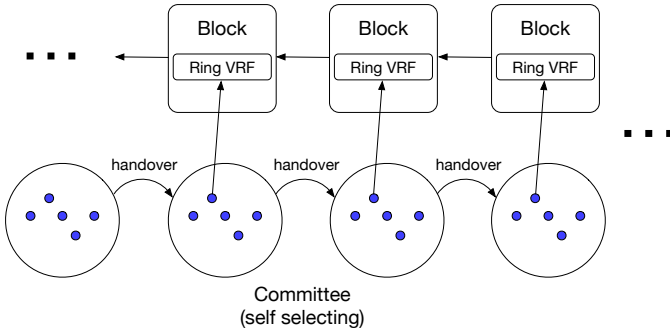


Fig. 2: Evolving committee overview

2) **Key Management:** As we mentioned above, all the key-related programs are deployed on a trusted execution environment (TEE). Specifically, on the initialization stage, a committee member loads the program Π_{DKG} , Π_{handover} and Π_{sign} into his/her TEE and generates an remote attestation σ which proves that the initialization process is correct. He/she then sends σ to the attestation service and gets the attestation verification report π . Finally, the attestation verification report π is put on the blockchain to be publicly verifiable. Note that merely using cryptographic tools cannot guarantee a malicious committee would faithfully delete his/her secret shares upon request. The deployment of TEE prevents the secret shares from being extracted and enforces key share erasure after the handover protocol, providing better confidentiality of secret keys.

3) **Universal Composability:** We will do security analysis under the Universally Composable (UC) [5], [6] model. In the UC framework, a protocol is represented by a set of interactive Turing machines (ITMs), and each ITM represents the program to be run by a participant. In the following, we assume that all ITMs are probabilistic polynomial time (PPT).

The security proof is based on the indistinguishability between the real/hybrid world and the ideal world. Let

$\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}$ be the execution of protocol Π in the real world with adversary $\mathcal{A}$ and environment $\mathcal{Z}$; let $\text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$ be the execution in the ideal world interacting with ideal functionality $\mathcal{F}$, simulator (ideal adversary) $\mathcal{S}$ and environment $\mathcal{Z}$. We say that Π UC-realizes $\mathcal{F}$ if for any PPT adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that no PPT environment $\mathcal{Z}$ can distinguish between $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}$ and $\text{EXEC}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$.

4) **UC Ideal Functionalities:** Our system utilizes the following UC functionalities as building blocks.

Blockchain. As shown in Fig. 3, the functionality $\mathcal{G}_{\text{blockchain}}$ consists of three interfaces: READ, WRITE and CHECK. Both broadcasting messages and modification of the balance are through the WRITE interface. There is a predicate Validate that ensures the validity of the blockchain content.

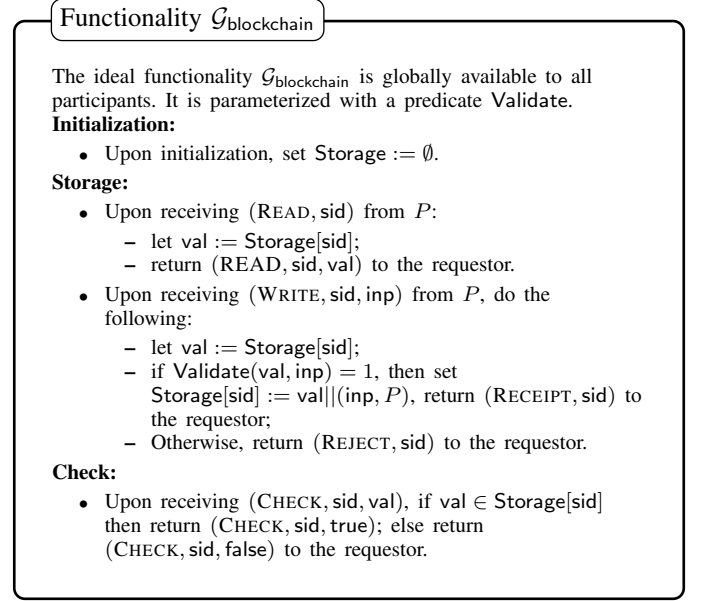


Fig. 3: The ideal blockchain functionality $\mathcal{G}_{\text{blockchain}}$

Distributed Key Generation. We propose a UC secure distributed key generation (DKG) protocol with identifiable aborts in Sec. 9. The DKG functionality is captured by $\mathcal{F}_{\text{DKG}}[\mathbb{G}]$. As shown in Fig. 4, the functionality chooses a random global secret gsk and shares it to the participants using Shamir's secret sharing. The adversary $\mathcal{S}$ is able to determine corrupted parties' secret shares, as, in the rushing adversary model, the corrupted parties can send messages after seeing the honest parties' messages in the real protocol.

Anonymous Cryptographic Sortition. The anonymous sortition functionality $\mathcal{F}_{\text{Anon-sortition}}^P$ is an abstraction that captures our Ring VRF-based sortition protocol, as depicted in Fig. 5. Unlike the conventional cryptographic sortition, when an elected committee member reveal his/her winning the sortition, he/she does not publish his/her identity. Instead, he/she can publish a message m_i through the REVEAL interface, which is an ephemeral public key in our scheme.

Handover. We design a UC secure handover protocol that allows the old committee to pass the shared secret keys to the new committee. Its functionality is captured by $\mathcal{F}_{\text{handover}}[\mathbb{G}]$ as depicted in Fig. 6, the adversary can determine corrupted parties' secret shares.

Functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$

The ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$ interacts with key generators $\mathcal{P} := \{P_1, \dots, P_n\}$, an ideal adversary $\mathcal{S}$. It's parameterized with threshold t . Denote $\mathcal{P}_c$ as the set of corrupted generators, $\mathcal{P}_h := \mathcal{P} \setminus \mathcal{P}_c$ as the set of honest generators, and $|\mathcal{P}_c| \leq t-1$. $\mathcal{F}_{\text{DKG}}$ maintains a set $\mathcal{N}$ (initially set to $\emptyset$).

- Upon receiving (KEYGEN, sid, P_i) from $P_i \in \mathcal{P}$, set $\mathcal{N} := \mathcal{N} \cup P_i$, send (KEYGENNOTIFY, sid, P_i) to $\mathcal{S}$. Wait until $|\mathcal{N}| = n$.
- Upon receiving (CORRUPTSHARES, sid, $(\{j, \text{psk}_j\}_{P_j \in \mathcal{P}_c})$) from $\mathcal{S}$:
 - Pick $\text{gsk} \leftarrow \mathbb{Z}_q$, and compute $\text{gpk} := g^{\text{gsk}}$;
 - Construct random polynomial $F(x) := \sum_{b=0}^{t-1} a_b \cdot x^b$ under the restriction $F(j) = \text{psk}_j$ for $P_j \in \mathcal{P}_c$, and $F(0) = \text{gsk}$;
 - Compute $\text{psk}_i := F(i)$ and $\text{ppk}_i = g^{\text{psk}_i}$ for $i \in [n]$;
 - Send (ABORT, sid) to $\mathcal{S}$ and wait for an answer(ABORT, sid, b);
 - Upon receiving the answer, if $b = 1$ then halt;
 - Otherwise, send (KEYGEN, sid, $(\{\text{ppk}_j\}_{j \in [n]})$) to $\mathcal{S}$ and send (KEYGEN, sid, $(\text{psk}_i, \{\text{ppk}_j\}_{j \in [n]})$) to $P_i, i \in [n]$.
- Upon receiving (READPK, sid) from any party, return (READPK, sid, gpk) to the requestor.

Fig. 4: DKG ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$

Functionality $\mathcal{F}_{\text{Anon-sortition}}^p$

The functionality $\mathcal{F}_{\text{Anon-sortition}}^p$ interacts with a set of parties $\mathcal{P} := \{P_1, \dots, P_n\}$ and adversary $\mathcal{S}$. It is parameterized with a variable $p \in [0, 1]$, and tables Pub and T . Let $\mathcal{P}_c$ be the set of corrupted parties, $\mathcal{P}_h := \mathcal{P} \setminus \mathcal{P}_c$ be the honest parties. Initially, set $\text{Pub} := \emptyset$, $T := \emptyset$.

Initialization:

- Upon receiving (INIT, sid) from the adversary $\mathcal{S}$, for each party P_i , pick a random bit $b_{P_i}^{\text{sid}}$ with $\Pr[b_{P_i}^{\text{sid}}] = p$ and set $T[\text{sid}] := \{ \langle P_i, b_{P_i}^{\text{sid}} \rangle \}_{P_i \in \mathcal{P}_c}$, and then send (INIT, sid, $T[\text{sid}]$) to the adversary $\mathcal{S}$.
- Upon receiving (REFRESH, sid) from $\mathcal{S}$, last step will be repeated.

Query:

- Upon receiving (QUERY, sid) from $P_i \in \mathcal{P}$, if $b_{P_i}^{\text{sid}}$ is initialized, send (QUERY, sid, $b_{P_i}^{\text{sid}}$) to the requestor.

Reveal:

- Upon receiving (REVEAL, sid, m_i) from $P_i \in \mathcal{P}$:
 - If $b_{P_i}^{\text{sid}} = 1$, then set $\text{Pub}[\text{sid}] := \text{Pub}[\text{sid}] \cup \{m_i\}$. Send (REVEAL, sid, m_i) to the adversary $\mathcal{S}$.
 - Otherwise, ignore the request.
- Upon receiving (STATE, sid) from any party $P_i \in \mathcal{P}$, send (STATE, sid, $\text{Pub}[\text{sid}]$) to the requestor.

Fig. 5: Anonymous cryptographic sortition ideal functionality

$\mathcal{F}_{\text{Anon-sortition}}^p$

Functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$

$\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ interacts with previous-epoch committee members $\mathcal{P} := \{P_1, \dots, P_{c_1}\}$, next-epoch committee members $\mathcal{Q} := \{Q_1, \dots, Q_{c_2}\}$ and adversary $\mathcal{S}$. It is parameterized with threshold t_1, t_2 , previous-epoch committee size c_1 , next-epoch committee size c_2 . Denote $\mathcal{P}_c$ as the set of previous-epoch corrupted parties, $\mathcal{P}_h := \mathcal{P} \setminus \mathcal{P}_c$ as the set of previous-epoch honest parties, and $|\mathcal{P}_c| \leq t_1 - 1$. Denote $\mathcal{Q}_c$ as the set of next-epoch corrupted parties, $\mathcal{Q}_h := \mathcal{Q} \setminus \mathcal{Q}_c$ as the set of next-epoch honest parties, and $|\mathcal{Q}_c| \leq t_2 - 1$. It maintains sets $\mathcal{M}, \mathcal{N}$ (initially set to $\emptyset$).

- Upon receiving (HANDOVER, sid, $\overline{\text{psk}}_i, \{\text{ppk}_j^{(i)}\}_{j \in [c_1]}$) from P_i :
 - Set $\mathcal{N} := \mathcal{N} \cup i$, send (HANDOVERNOTIFY, sid, P_i) to $\mathcal{S}$. Wait until $|\mathcal{N}| = c_1$;
 - Set $\{\text{ppk}_j\}_{j \in [c_1]}$ as the majority of $\{\{\text{ppk}_j^{(i)}\}_{j \in [c_1]}\}_{i \in [c_1]}$;
 - Set $\mathcal{M}$ as the first t_1 parties that satisfy $\text{ppk}_j = g^{\text{psk}_j}$, set $\text{psk}_j := \text{psk}_j$.
- Upon receiving (CORRUPTSHARES, sid, $(\{j, \text{npsk}_j\}_{Q_j \in \mathcal{Q}_c})$) from $\mathcal{S}$:
 - Compute $\text{gsk} = \prod_{j \in \mathcal{M}} \lambda_j \cdot \text{psk}_j$, where $\{\lambda_j\}_{j \in \mathcal{M}}$ are Lagrange coefficients, i.e., $\lambda_j := \prod_{\ell \in \mathcal{M} \setminus \{j\}} \frac{\ell}{\ell - j}$;
 - Construct random polynomial $F(x) := \sum_{b=0}^{t_2-1} a_b \cdot x^b$ under the restriction $F(j) = \text{npsk}_j$ for $Q_j \in \mathcal{Q}_c$, and $F(0) = \text{gsk}$;
 - Compute $\text{npsk}_i := F(i)$ and $\text{nppk}_i = g^{\text{npsk}_i}$ for $i \in [c_2]$;
 - Send (ABORT, sid) to $\mathcal{S}$ and wait for an answer(ABORT, sid, b);
 - Upon receiving the answer, if $b = 1$ then halt;
 - Otherwise, send (HANDOVER, sid, $(\{\text{nppk}_j\}_{j \in [c_2]})$) to $\mathcal{S}$ and (HANDOVER, sid, $(\text{npsk}_i, \{\text{nppk}_j\}_{j \in [c_2]})$) to $Q_i, i \in [c_2]$.

Fig. 6: Handover ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$

D. Evolving Committee

This section will describe the evolving committee secret sharing in detail, as well as Ring VRF $\Pi_{\text{Ring-VRF}}$ and zk proof for decryption correctness Π_{Dec} as building blocks.

1) *Ring VRF*: In this section, we introduce a new notion, called *Ring Verifiable Random Function* (Ring-VRF). Given a set of public keys $\{\text{pk}_0, \dots, \text{pk}_{N-1}\}$ and an input $x \in \{0, 1\}^*$, R-VRF allows the user to invoke its private key sk to generate $v \leftarrow \text{PRF}_{\text{sk}}(x)$, and convince the public that

$$\mathcal{R}_{\text{VRF}} = \left\{ \begin{array}{l} ((\text{pk}_0, \dots, \text{pk}_{N-1}, x, v), \text{sk}, \ell) \mid \\ \ell \in \{0, \dots, N-1\} \wedge (\text{pk}_\ell, \text{sk}) \in \mathcal{R}_{\text{pk}} \wedge \\ v \leftarrow \text{PRF}_{\text{sk}}(x) \end{array} \right\}$$

where $\mathcal{R}_{\text{pk}} = \{(\text{pk}, \text{sk}) \mid \text{pk} = g^{\text{sk}}\}$.

We give a formal definition of Ring VRF as follows. It consists of 3 PPT algorithms.

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\lambda)$: The key generation algorithm produces a public key pk and a secret key sk .
- $(v, \pi) \leftarrow \text{Eval}(\text{sk}, x, \{\text{pk}_0, \dots, \text{pk}_{N-1}\})$: The evaluation algorithm takes as input secret key sk , x and a set of public keys $\{\text{pk}_0, \dots, \text{pk}_{N-1}\}$. It outputs v and a proof π for $\mathcal{R}_{\text{VRF}}$.
- $0/1 \leftarrow \text{Verify}(x, v, \pi)$: The verifier algorithm takes as input (x, v, π) and outputs 0 or 1.

- **PRF:** Let $\mathbb{G}$ be a cyclic group with prime order q , and denote g as the generator. $H : \{0, 1\}^* \mapsto \mathbb{G}$ is a hash function that maps an arbitrary length string to a group element. In this work, we adopt PRF $\text{PRF} : \{0, 1\}^* \times \mathbb{Z}_q \mapsto \mathbb{G}$ as follows: $\text{PRF}_{\text{sk}}(x) := H(x)^{\text{sk}}$. Note that, in the actual implementation, H can be implemented by the “Elligator 2” mapping proposed in [3].
 - **Our construction:** We first describe the intuition of the protocol. In the first step of our protocol, the prover create a new commitment of sk , denoted as $c = \text{Com}_{\text{ck}}(\text{sk}; t)$. Then, the protocol can be viewed as two parts run in parallel. One part is inspired by one-out-of-many protocol [10] that ensures pk_ℓ is in the ring. The other part ensures that c and v corresponds to the same sk . Combining the two parts together we get the protocol for Ring VRF. In practice, Pedersen commitment can be used as the additive homomorphic commitment scheme, i.e., $\text{Com}_{\text{ck}}(a; r) = g^a h^r$.
- In this paragraph we specify the polynomial $p_i(e)$ used in the protocol. Following the idea of [10], we write $i = i_1 \dots i_n$ and $\ell = \ell_1 \dots \ell_n$ in binary, and we let $\delta_{i\ell}$ be Kronecker’s delta, i.e., $\delta_{i\ell} = 1$ and $\delta_{i\ell} = 0$ for $i \neq \ell$. We let $f_j = \ell_j e + a_j$, let $f_{j,1} = f_j = \ell_j e + a_j = \delta_{1\ell_j} e + a_j$ and $f_{j,0} = e - f_j = (1 - \ell_j)e - a_j = \delta_{0\ell_j} e - a_j$. Then, $p_i(e) = \prod_{j=1}^n f_{j,i_j}$ has the form:

$$p_i(e) = \prod_{j=1}^n (\delta_{i_j \ell_j} e) + \prod_{k=0}^{n-1} p_{i,k} e^k = \delta_{i\ell} e^n + \prod_{k=0}^{n-1} p_{i,k} e^k \quad (1)$$

Finally, Fig. 7 shows the Sigma protocol for the relation $\mathcal{R}_{\text{VRF}}$. By Fiat-Shamir heuristic [8] we can transform it into a non-interactive zero knowledge proof. Moreover, we add a tag in the hash input, which is the ephemeral public key in our application scenario.

Theorem 1. *Let Com be an additive homomorphic non-interactive commitment scheme that is perfectly hiding and computationally binding with adversarial advantage ε . Protocol $\Pi_{\text{zk-VRF}}$ as described in Fig. 7 for relation $\mathcal{R}_{\text{VRF}}$ is a three-move public coin zero-knowledge protocol with perfect completeness, $(n+1)$ -special soundness with adversarial advantage ε and perfect special honest verifier zero-knowledge.*

Proof. To prove that protocol $\Pi_{\text{Ring-VRF}}$ is perfect complete observe that $\prod_{j=1}^n f_{j,i_j}$ is a polynomial in the challenge e of the form $p_i(e) = \delta_{i\ell} e^n + \prod_{k=0}^{n-1} p_{i,k} e^k$. When $\text{pk}_\ell = g^{\text{sk}}$, c_ℓ is a commitment to 0 we therefore get that $c_\ell^{\prod_{j=1}^n f_{j,\ell_j}}$ in the verification equation is a commitment to 0. Since $c_{d_k} = \prod_i c_i^{p_{i,k}} \text{Com}_{\text{ck}}(0; \rho_k)$, by the additive homomorphic property of Com , the terms involving $p_{i,k}$ are all cancelled. Hence, the verification equation $\prod_i c_i^{\prod_{j=1}^n f_{j,i_j}} \cdot \prod_{k=0}^{n-1} c_{d_k}^{-e^k} = \text{Com}_{\text{ck}}(0; z_d)$ always holds. And $g^{y_1} h^{y_2} = m_1 c^e, u^{y_1} = m_2 v^e$ also hold. Therefore the protocol is complete.

To prove that protocol $\Pi_{\text{Ring-VRF}}$ is $(n+1)$ -sound, we show that an adversary with probability ε of breaking $(n+1)$ -soundness can be converted into an adversary

Sigma protocol for Ring VRF $\Pi_{\text{Ring-VRF}}$

CRS: the commitment key $\text{ck} = h$;

Statement: $\text{pk}_0, \dots, \text{pk}_{N-1}, x, v$;

Witness: sk, ℓ such that $\ell \in \{0, \dots, N-1\} \wedge \text{pk}_\ell = g^{\text{sk}} \wedge v = H(x)^{\text{sk}}$.

Prover:

- $t \leftarrow \mathbb{Z}_q$, $c = g^{\text{sk}} h^t$;
- For $i = 0, \dots, N-1$, compute $c_i = \text{pk}_i / c$;
- For $j = 1, \dots, n$
 - * $r_j, a_j, s_j, t_j, \rho_k \leftarrow \mathbb{Z}_q$;
 - * $c_{\ell_j} = \text{Com}_{\text{ck}}(\ell_j; r_j)$;
 - * $c_{a_j} = \text{Com}_{\text{ck}}(a_j; s_j)$;
 - * $c_{b_j} = \text{Com}_{\text{ck}}(\ell_j a_j; t_j)$;
 - * $c_{d_k} = \prod_i c_i^{p_{i,k}} \text{Com}_{\text{ck}}(0; \rho_k)$, using $k = j-1$ and $p_{i,k}$ from Eq. 1;
- $s', t' \leftarrow \mathbb{Z}_q$, $m_1 = g^{s'} h^{t'}$, $m_2 = u^{s'}$, using $u = H(x)$;
- $P \rightarrow V$:
 $(c, c_{\ell_1}, c_{a_1}, c_{b_1}, c_{d_0}, \dots, c_{\ell_n}, c_{a_n}, c_{b_n}, c_{d_{n-1}}, m_1, m_2)$.

Verifier:

- $e \leftarrow \mathbb{Z}_q$;
- In the non-interactive version, $e = \text{hash}(\text{tag}, \text{ck}, \text{pk}_0, \dots, \text{pk}_{N-1}, r, x, c, c_{\ell_1}, c_{a_1}, c_{b_1}, c_{d_0}, \dots, c_{\ell_n}, c_{a_n}, c_{b_n}, c_{d_{n-1}}, m_1, m_2)$.
- $V \rightarrow P$: e .

Prover:

- For $j = 1, \dots, n$
 - * $f_j = \ell_j e + a_j$;
 - * $z_{a_j} = r_j e + s_j$;
 - * $z_{b_j} = r_j (e - f_j) + t_j$;
 - * $z_d = (-t) e^n - \sum_{k=0}^{n-1} \rho_k e^k$;
- $y_1 = s' + \text{sk} \cdot e$, $y_2 = t' + t e$;
- $P \rightarrow V$: $(f_1, z_{a_1}, z_{b_1}, \dots, f_n, z_{a_n}, z_{b_n}, z_d, y_1, y_2)$.

Verifier:

- For $i = 0, \dots, N-1$, compute $c_i = \text{pk}_i / c$;
- For all $j \in \{1, \dots, n\}$, check
 - * $c_{\ell_j}^e c_{a_j} = \text{Com}_{\text{ck}}(f_j; z_{a_j})$;
 - * $c_{\ell_j}^{e-f_j} c_{b_j} = \text{Com}_{\text{ck}}(0; z_{b_j})$;
- check $\prod_i c_i^{\prod_{j=1}^n f_{j,i_j}} \cdot \prod_{k=0}^{n-1} c_{d_k}^{-e^k} = \text{Com}_{\text{ck}}(0; z_d)$, using $f_{j,1} = f_j$ and $f_{j,0} = e - f_j$;
- check $g^{y_1} h^{y_2} = m_1 c^e$, $u^{y_1} = m_2 v^e$, using $u = H(x)$;
- Output 1 iff all the checks pass.

Fig. 7: Sigma protocol for Ring VRF $\Pi_{\text{Ring-VRF}}$

that has probability ε of breaking the binding property of the commitment scheme.

Suppose the cheating prover creates $n+1$ accepting responses $f_1^{(0)}, \dots, y_2^{(0)}, \dots, y_2^{(n)}, \dots, z_d^{(n)}$ to $n+1$ different challenges $e^{(0)}, \dots, e^{(n)}$ on the same initial message $c, c_{\ell_1}, \dots, m_2$, we first show that $\ell_j \in \{0, 1\}$. Pick two responses $f_j^{(0)}, z_{a_j}^{(0)}, z_{b_j}^{(0)}$ and $f_j^{(1)}, z_{a_j}^{(1)}, z_{b_j}^{(1)}$ to challenges $e^{(0)}, e^{(1)}$ on the commitments c_{a_j}, c_{b_j} . By combining the verification equations we get $c_{\ell_j}^{e^{(0)}-e^{(1)}} = \text{Com}_{\text{ck}}(f_j^{(0)} - f_j^{(1)}; z_{a_j}^{(0)} - z_{a_j}^{(1)})$ and $c_{\ell_j}^{e^{(0)}-f_j^{(0)}-e^{(1)}+f_j^{(1)}} = \text{Com}_{\text{ck}}(0; z_{b_j}^{(0)} - z_{b_j}^{(1)})$. Defining $\gamma_j = \frac{f_j^{(0)} - f_j^{(1)}}{e^{(0)} - e^{(1)}}$ and $\gamma_j = \frac{z_{a_j}^{(0)} - z_{a_j}^{(1)}}{e^{(0)} - e^{(1)}}$ we extract an opening of $c_{\ell_j} = \text{Com}_{\text{ck}}(\ell_j; \gamma_j)$. Furthermore, since $c_{\ell_j}^{e^{(0)}-f_j^{(0)}-e^{(1)}+f_j^{(1)}} =$

$c_{\ell_j}^{(1-\ell_j)(e^{(0)}-e^{(1)})} = \text{Com}_{\text{ck}}(\ell_j(1-\ell_j)(e^{(0)}-e^{(1)}); \gamma_j(1-\ell_j)(e^{(0)}-e^{(1)})) = \text{Com}_{\text{ck}}(0; z_{b_j}^{(0)} - z_{b_j}^{(1)})$, we either get a breach of the binding property of the commitment scheme or we have $\ell_j(1-\ell_j) = 0$, which implies $\ell_j \in \{0, 1\}$. Let a_j be the number committed in c_{a_j} , from the verification equation $c_{\ell_j}^e c_{a_j} = \text{Com}_{\text{ck}}(f_j; z_{a_j})$ we get that $f_j^{(0)} = \ell_j e^{(0)} + a_j, \dots, f_j^{(n)} = \ell_j e^{(n)} + a_j$ for all $j = 1, \dots, n$ unless the adversary breaks the binding property of the commitment scheme, it must hold for all challenges. From $f_{j,1} = f_j$ and $f_{j,0} = e - f_j$ we get $f_{j,1} = \ell_j e + a_j$ and $f_{j,0} = (1-\ell_j)e - a_j$. For $i \neq \ell$ we therefore get that $\prod_{j=1}^n f_{j,i_j}$ is a degree at most $n-1$ polynomial $p_i(e)$, and for $i = \ell$ it is a polynomial of the form $p_\ell(e) = e^n + \dots$. This means we can rewrite $\prod_i c_i^{\prod_{j=1}^n f_{j,i_j}} \cdot \prod_{k=0}^{n-1} c_{d_k}^{-e^k} = \text{Com}_{\text{ck}}(0; z_d)$ as

$$c_\ell^{e^n} \cdot \prod_{k=0}^{n-1} c_{*k}^{e^k} = \text{Com}_{\text{ck}}(0; z_d)$$

for some fixed $c_{*0}, \dots, c_{*n-1}$, which can be computed from commitments in the statement and the initial message.

Observe that the vectors $(1, e^{(\beta)}, \dots, (e^{(\beta)})^n), \beta = 0, \dots, n$ can be viewed as rows in a Vandermonde matrix because $e^{(0)}, \dots, e^{(n)}$ are all different. The matrix is invertible and we can therefore find a linear combination $(\alpha_0, \dots, \alpha_n)$ of the rows that gives us the vector $(0, \dots, 0, 1)$. Combining the $n+1$ accepting verification equations we therefore get

$$c_\ell = \prod_{\beta=0}^n (c_\ell^{(e^{(\beta)})^n} \cdot \prod_{k=0}^{n-1} c_{*k}^{(e^{(\beta)})^k})^{\alpha_\beta} = \text{Com}_{\text{ck}}(0; \sum_{\beta=0}^n \alpha_\beta z_d^{(\beta)})$$

This gives us an extracted opening of c_ℓ to 0. Since $c_i = \text{pk}_i/c$, denote $\text{pk}_\ell = g^{\text{sk}_\ell}$ we have $c = g^{\text{sk}_\ell} h^t$.

As to the PRF part, to show c and v corresponds to the same $\text{sk} = \text{sk}_\ell$, we pick two responses $y_1^{(0)}, y_2^{(0)}, y_1^{(1)}, y_2^{(1)}$ to the challenges $e^{(0)}, e^{(1)}$. We have $g^{y_1^{(0)}} h^{y_2^{(0)}} = m_1 c^{e^{(0)}}$, $g^{y_1^{(1)}} h^{y_2^{(1)}} = m_1 c^{e^{(1)}}$. Divide the two equations we get $c = g^{(y_1^{(0)} - y_1^{(1)})(e^{(0)} - e^{(1)})^{-1}} h^{(y_2^{(0)} - y_2^{(1)})(e^{(0)} - e^{(1)})^{-1}}$. Similarly, we have $u^{y_1^{(0)}} = m_2 v^{e^{(0)}}$, $u^{y_1^{(1)}} = m_2 v^{e^{(1)}}$. Divide the two equations we get $v = u^{(y_1^{(0)} - y_1^{(1)})(e^{(0)} - e^{(1)})^{-1}}$. Thus we extract $\text{sk} = (y_1^{(0)} - y_1^{(1)})(e^{(0)} - e^{(1)})^{-1}$ and $t = (y_2^{(0)} - y_2^{(1)})(e^{(0)} - e^{(1)})^{-1}$.

To prove that protocol $\Pi_{\text{Ring-VRF}}$ is perfect special honest verifier zero-knowledge, we build a special honest verifier zero-knowledge simulator that is given a challenge $e \in \{0, 1\}^\lambda$. First, it picks the elements of the response uniformly at random as $f_1, \dots, y_2 \leftarrow \mathbb{Z}_q$. It picks $c = g^a h^r$ at random and computes $c_i = \text{pk}_i/c$. It picks $c_{\ell_1}, \dots, c_{\ell_n}, c_{d_1}, \dots, c_{d_{n-1}} \leftarrow \text{Com}_{\text{ck}}(0)$ as random commitments to 0. Then, it computes $c_{a_j} = c_{\ell_j}^{-x} \text{Com}_{\text{ck}}(f_j; z_{a_j})$, $c_{b_j} = c_{\ell_j}^{f_j-x} \text{Com}_{\text{ck}}(0; z_{b_j})$, $c_{d_0} = \prod_i c_i^{\prod_{j=1}^n f_{j,i_j}} \cdot \prod_{k=1}^{n-1} c_{d_k}^{-x^k} \cdot \text{Com}_{\text{ck}}(0; -z_d)$, $m_1 = g^{y_1} h^{y_2} c^{-e}$, $m_2 = u^{y_1} v^{-e}$. Finally, it returns the simulated initial message and response $(c, c_{\ell_1}, \dots, m_2, f_1, \dots, y_2)$.

Now we argue that the simulated proof and the real proof are perfectly indistinguishable. First, in both real proofs and simulated proofs $f_1, \dots, y_2$ are uniformly random in $\mathbb{Z}_q$. Furthermore, by the verification equations, $c_{a_1}, c_{b_1}, \dots, c_{a_n}, c_{b_n}, c_{d_0}, m_1, m_2$ are determined by on $f_1, \dots, y_2$ and $c, c_{\ell_1}, \dots, c_{d_{n-1}}$ both in real and in simulated proofs. Since the commitment scheme Com is perfectly hiding, the adversary cannot distinguish between the real and simulated commitments $c, c_{\ell_1}, \dots, c_{\ell_n}, c_{d_1}, \dots, c_{d_{n-1}}$. Hence, the protocol is perfect zero-knowledge. $\square$

2) *ZK Proof for Hybrid Decryption*: In this section, we construct a Sigma protocol to show the correctness of decryption w.r.t. the hybrid encryption scheme, and then use Fiat-Shamir heuristic to transform it into a non-interactive version. The non-interactive zero knowledge protocol will be used later in our DKG protocol and handover protocol. To prove the correctness of decryption, actually, the prover only needs to provide $c_2 = (c_1)^{\text{sk}}$. The verifier can then computes $k \leftarrow \text{hash}(c_2)$ and checks the decryption by $m = \text{AES-GCM}_k^{-1}(u)$ (cf. II-B1). The Sigma protocol is depicted in Fig. 8. Similar to Ring VRF, by Fiat-Shamir heuristic we can transform it into a non-interactive zero knowledge proof. We denote the NIZK protocol as $\Pi_{\text{NIZK-Dec}}$.

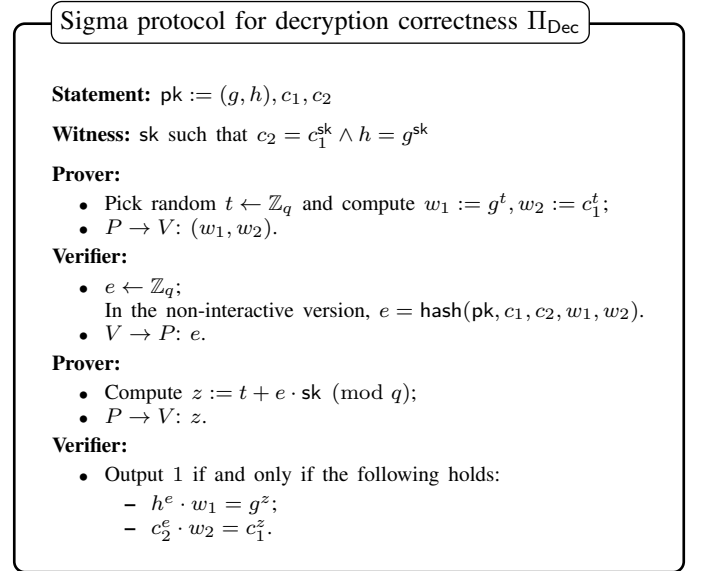


Fig. 8: Sigma protocol for decryption correctness Π_{Dec}

Theorem 2. *The Sigma protocol in Fig. 8 for relation $\mathcal{R}$ is perfectly complete, 2-special sound and perfectly special honest verifier zero-knowledge.*

Proof. To see that the Sigma protocol is perfectly complete observe that $c_2 = c_1^{\text{sk}}$ and $h = g^{\text{sk}}$. Thus we have $g^z = g^t \cdot g^{e \cdot \text{sk}} = g^t \cdot (g^{\text{sk}})^e = h^e \cdot w_1$ and $c_1^z = c_1^t \cdot c_1^{e \cdot \text{sk}} = c_1^t \cdot (c_1^{\text{sk}})^e = c_2^e \cdot w_2$.

To see that the Sigma protocol is 2-special sound, we assume the prover respond to two different challenges e, e' with z, z' . Divide the verification equation we get $g^{z-z'} = h^{e-e'}$ and $c_1^{z-z'} = c_2^{e-e'}$. From $g^{z-z'} = h^{e-e'}$ we extract

$sk = (z - z')(e - e')^{-1}$. Combining it with $c_1^{z-z'} = c_2^{e-e'}$ we can prove that $c_2 = c_1^{sk}$.

To see that the Sigma protocol is perfectly special honest verifier zero-knowledge we build a special honest verifier zero-knowledge simulator that is given a challenge e . It picks $z \leftarrow \mathbb{Z}_p$, then computes $w_1 = g^z/h^e$ and $w_2 = c_1^z/c_2^e$. We can see that for a real statement, i.e., $c_2 = c_1^{sk}$, we have $w_2 = w_1^r$ for both real proof and simulated proof, which means real proof and simulated proof have the same distribution. $\square$

3) *Distributed Key Generation*: For distributed key generation, we modify the distributed key generation protocol by Gennaro and Goldfeder [9] to achieve UC secure. We use a NIZK proof and split the broadcast step into two phase. In phase 1 parties broadcast the ciphertexts; In phase 2 parties broadcast the verification components. This allows the ideal world simulator to compute the verification components reversely to achieve indistinguishability. The protocol $\Pi_{DKG}^{t,n}$ is depicted in Fig. 9.

Distributed key generation protocol $\Pi_{DKG}^{t,n}[\mathbb{G}]$

Setup: Each party $P_i \in \mathcal{P}$ is associated with a ephemeral public key epk_i , and P_i holds the corresponding ephemeral secret key esk_i .

- Upon receiving (KEYGEN, sid, P_i) from $\mathcal{Z}$, $P_i \in \mathcal{P}$ does the following:
 - Select $u_i \leftarrow \mathbb{Z}_q$ and compute $c_i = \text{Com}(g^{u_i})$, where $\text{Com}(g^{u_i}) = \text{hash}(g^{u_i}; r_i)$. Send (WRITE, sid, c_i) to $\mathcal{G}_{\text{blockchain}}$. Wait until all parties have broadcast c_i ;
 - Choose a random degree- $(t-1)$ polynomial G_i with $G_i(0) = u_i$;
 - For each $k \in [n]$, Set $\sigma_{i,k} = G_i(k)$ and use the k 's ephemeral public key to encrypt it with hybrid encryption scheme, setting $ct_{i,k} = \text{Enc}_{epk_k}(\sigma_{i,k})$;
 - Let the coefficients of $G_i(x)$ be $a_{i,0}, \dots, a_{i,t-1}$. Compute $A_{i,0} = g^{a_{i,0}} = g^{u_i}, \dots, A_{i,t-1} = g^{a_{i,t-1}}$;
 - Send (WRITE, sid, $(ct_{i,1}, \dots, ct_{i,n})$) to $\mathcal{G}_{\text{blockchain}}$;
 - Send (READ, sid) to $\mathcal{G}_{\text{blockchain}}$ and use esk_i to decrypt them to get $\{\sigma_{j,i}\}_{j \in [n] \setminus \{i\}}$;
 - Send (WRITE, sid, $((g^{u_i}; r_i), A_{i,1}, \dots, A_{i,t-1})$) to $\mathcal{G}_{\text{blockchain}}$ ($G_i(0) = u_i$ implies $A_{i,0} = g^{u_i}$);
 - Send (READ, sid) to $\mathcal{G}_{\text{blockchain}}$ to get $\{(g^{u_j}; r_j), A_{j,1}, \dots, A_{j,t-1}\}_{j \in [n] \setminus \{i\}}$;
 - Check if $c_i = \text{hash}(g^{u_i}; r_i)$;
 - Check if the shares are right, i.e., $g^{\sigma_{k,i}} = \prod_{j=0}^{t-1} (A_{k,j})^{i^j}$. If not, the receiver will generate a NIZK proof π to prove that he receives the wrong share (cf. non-interactive version of Fig. 8) and send (WRITE, sid, π) to $\mathcal{G}_{\text{blockchain}}$. (One valid complaint against P_j will disqualify P_j , and the protocol will abort.);
 - Compute secret key share $psk_i = \sum_{j \in [n]} \sigma_{j,i}$;
 - Set public key as $gpk = \prod_{j \in [n]} g^{u_j}$, secret key as $gsk = \sum_{j \in [n]} u_j$, each party's partial public key as $ppk_j = g^{x_j} = \prod_{\ell \in [n]} g^{\sigma_{\ell,j}} = \prod_{\ell \in [n]} (\prod_{k=0}^{t-1} (A_{\ell,k})^{j^k})$;
 - Return (KEYGEN, sid, $(psk_i, \{ppk_j\}_{j \in [n]})$) to $\mathcal{Z}$.
- Upon receiving (READPK, sid) from $\mathcal{Z}$, return (READPK, sid, gpk) to $\mathcal{Z}$.

Fig. 9: Distributed key generation protocol $\Pi_{DKG}^{t,n}[\mathbb{G}]$

4) *Anonymous Committee Sortition*: In the committee sortition process, we hope to hide the committee members so as to

avoid coercion or DoS attack. The committee sortition protocol works as follows. For protocol setup, each party owns his/her secret key sk and there is an agreed-upon random number x from the blockchain. At the beginning of the protocol, each party computes $v = \text{PRF}_{sk}(x)$ and check if v is less than a certain threshold T , which represents his/her winning the sortition. The sortition winners will broadcast a Ring VRF proof π along with his/her ephemeral public key epk . Other members watch the blockchain and record the valid ephemeral public keys.

The ideal functionality $\mathcal{F}_{\text{Anon-sortition}}^p$ can be realized by a Ring VRF-based sortition protocol in the $\{\mathcal{F}_{\text{RO}}, \mathcal{G}_{\text{blockchain}}\}$ -hybrid model. The simulator $\mathcal{S}$ performs as follows: Upon receiving (INIT, sid, $T[\text{sid}]$) from $\mathcal{F}_{\text{Anon-sortition}}^p$, $\mathcal{S}$ simulates $\mathcal{F}_{\text{RO}}$ so that $v_i < T$ for $i : b_{P_i}^{\text{sid}} = 1$. Upon receiving (REVEAL, sid, m_i) from $\mathcal{F}_{\text{Anon-sortition}}^p$, $\mathcal{S}$ simulates a Ring VRF proof π and sends (WRITE, sid, (π, m_i)) to $\mathcal{G}_{\text{blockchain}}$. Since $\mathcal{F}_{\text{RO}}$ is simulated by $\mathcal{S}$, the real world and ideal world are indistinguishable.

E. Handover

The handover protocol utilizes the Shamir secret sharing scheme to re-share the secret keys from the previous committee to the new committee. In particular, it uses Feldman's VSS to achieve verifiability and accountability. Similar to the DKG protocol, a NIZK proof for decryption correctness is used to identify the malicious behaviors and the broadcast step is split into two phase to ensure UC security. The detail of handover protocol is depicted in Fig. 10.

Note that, our handover protocol achieves identifiable abort. Namely, if a malicious committee member does not follow the protocol, the honest party will raise a complain against him/her and he/she will be disqualified. Then, the remaining parties will re-run the protocol so the the protocol will eventually terminate. In DeepSafe, these disqualified committee members will be punished when the handover protocol finishes.

1) *Threshold Signature*: For threshold signature, we support common signature schemes used in blockchain cryptocurrencies, including BLS [1], Schnorr [18] and ECDSA [11]. To generate a BLS signature, the committee only needs to reconstruct the secret in the exponent. A Schnorr signature can be generated by the additive homomorphism property of Shamir secret sharing. As to the most difficult ECDSA, we adopt the threshold signature protocol proposed by Gennaro and Goldfeder [9], which achieves one round threshold ECDSA with identifiable abort.

F. Security analysis

In this section, we will formally prove that our DKG protocol $\Pi_{DKG}^{t,n}[\mathbb{G}]$ and handover protocol $\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ UC-realize DKG ideal functionality $\mathcal{F}_{DKG}^{t,n}[\mathbb{G}]$ and handover ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$, respectively.

Theorem 3. *Let the hybrid encryption scheme HEnc be IND-CPA secure with adversary advantage $\text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$. Let the NIZK protocol for decryption correctness $\Pi_{\text{NIZK-Dec}}$ be statistically sound with soundness error $\text{Adv}_{\text{NIZK-Dec}}^{\text{sound}}(\mathcal{A}, \lambda)$*

The handover protocol $\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$

Setup: Each next-epoch committee member $Q_i \in \mathcal{Q}$ is associated with an ephemeral public key epk_i , and Q_i holds the corresponding ephemeral secret key esk_i . The protocol is parameterized with threshold t_1, t_2 , previous-epoch committee size c_1 , next-epoch committee size c_2 . The shares held by $\mathcal{C}_1$ define a degree- $(t_1 - 1)$ polynomial F_1 with $F_1(0) = \sigma$, where each seat j holds a share $\text{psk}_j = F_1(j)$. All parties agree on the non-zero evaluations points $I = \{1, 2, \dots, c_1\}$ used for a t_1 -of- c_1 Shamir secret-sharing scheme.

Upon receiving $(\text{HANDOVER}, \text{sid}, \overline{\text{psk}}_i, \{\text{ppk}_j\}_{j \in [c_1]})$ from $\mathcal{Z}$:

- Previous-epoch committee member $P_i \in \mathcal{P}$ does the following:
 - Send $(\text{WRITE}, \text{sid}, \{\text{ppk}_j^{(i)}\}_{j \in [c_1]})$ to $\mathcal{G}_{\text{blockchain}}$;
 - Send $(\text{READ}, \text{sid})$ to $\mathcal{G}_{\text{blockchain}}$ and set $\{\text{ppk}_j\}_{j \in [c_1]}$ as the majority of $\{\{\text{ppk}_j^{(k)}\}_{j \in [c_1]}\}_{k \in [c_1]}$;
 - Choose a random degree- $(t_2 - 1)$ polynomial G_i with $G_i(0) = \text{psk}_j$;
 - For each $k \in [c_2]$, Set $\sigma_{i,k} = G_i(k)$ and use the k 's ephemeral public key to encrypt it with hybrid encryption scheme, setting $\text{ct}_{i,k} = \text{Enc}_{\text{epk}_k}(\sigma_{i,k})$;
 - Let the coefficients of $G_i(x)$ be $a_{i,0}, \dots, a_{i,t_2-1}$. Compute $A_{i,1} = g^{a_{i,1}}, \dots, A_{i,t_2-1} = g^{a_{i,t_2-1}}$. ($A_{i,0} = \text{ppk}_i$, which is publicly known, so there is no need to broadcast it);
 - Send $(\text{WRITE}, \text{sid}, (\text{ct}_{i,1}, \dots, \text{ct}_{i,c_2}))$ to $\mathcal{G}_{\text{blockchain}}$. Wait until all parties have broadcast $(\text{ct}_{j,1}, \dots, \text{ct}_{j,c_2})$;
 - Send $(\text{WRITE}, \text{sid}, (A_{i,1}, \dots, A_{i,t_2-1}))$ to $\mathcal{G}_{\text{blockchain}}$.
- Next-epoch committee member $Q_i \in \mathcal{Q}$ does the following:
 - Send $(\text{READ}, \text{sid})$ to $\mathcal{G}_{\text{blockchain}}$ and use esk to decrypt the ciphertexts to get $\{\sigma_{j,i}\}_{j \in [c_1]}$;
 - Send $(\text{READ}, \text{sid})$ to $\mathcal{G}_{\text{blockchain}}$ to get $\{A_{j,k}\}_{j \in [c_1]}$;
 - Check if the shares are right, i.e., $g^{\sigma_{j,k}} = \prod_{n=0}^{t_2-1} (A_{j,n})^{k^n}$, where $A_{j,0} = \text{ppk}_j$. If not, the receiver will broadcast a NIZK proof to prove that he receives the wrong share (cf. non-interactive version of Fig. 8); (One valid complain against P_j will disqualify P_j and the protocol will abort.)
 - Let the indices of the first t_1 valid messages be set $\mathcal{I}$, $\mathcal{I} \subseteq [c_1]$ and $|\mathcal{I}| = t_1$. Compute the share of the global secret corresponding to seat i as $\text{npsk}_i = \sum_{j \in \mathcal{I}} \lambda_j \cdot \sigma_{j,i}$, where $\lambda_j := \prod_{\ell \in \mathcal{I} \setminus \{j\}} \frac{\ell}{\ell - j}$;
 - Compute each new committee member's partial public key as $\text{nppk}_k = \prod_{j \in \mathcal{I}} (g^{\sigma_{j,k}})^{\lambda_j} = \prod_{j \in \mathcal{I}} (\prod_{n=0}^{t_2-1} (A_{j,n})^{k^n})^{\lambda_j}$, $k = 1, \dots, c_2$, where $\lambda_j := \prod_{\ell \in \mathcal{I} \setminus \{j\}} \frac{\ell}{\ell - j}$;
 - Return $(\text{HANDOVER}, \text{sid}, (\text{npsk}_i, \{\text{nppk}_j\}_{j \in [c_2]}))$ to $\mathcal{Z}$.

Fig. 10: The handover protocol $\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$

and perfect zero-knowledge. The protocol $\Pi_{\text{DKG}}^{t,n}[\mathbb{G}]$ described in fig. 9 UC-realized $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$ against static corruption up to $t - 1$ parties in the $\{\mathcal{G}_{\text{blockchain}}, \mathcal{F}_{\text{RO}}\}$ -hybrid model with distinguishing advantage

$$n^2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) + n(t - 1) \cdot \text{Adv}_{\text{NIZK}_{\text{Dec}}}^{\text{sound}}(\mathcal{A}, \lambda).$$

Proof. We first construct a simulator $\mathcal{S}$ such that any non-uniform PPT environment $\mathcal{Z}$ cannot distinguish between the ideal world execution $\text{EXEC}_{\Pi_{\text{DKG}}^{t,n}[\mathbb{G}], \mathcal{S}, \mathcal{Z}}$ and the real world execution $\text{EXEC}_{\Pi_{\text{DKG}}^{t,n}[\mathbb{G}], \mathcal{A}, \mathcal{Z}}$. In the ideal world, the parties

interact with functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$ and the corrupted parties are controlled by $\mathcal{S}$; in the real world, the parties $\mathcal{P} = \{P_1, \dots, P_n\}$ run protocol $\Pi_{\text{DKG}}^{t,n}[\mathbb{G}]$ in the $\{\mathcal{G}_{\text{blockchain}}, \mathcal{F}_{\text{RO}}\}$ -hybrid world and the corrupted parties are controlled by a dummy adversary $\mathcal{A}$ who simply forwards messages from/to $\mathcal{Z}$.

Simulator. The simulator $\mathcal{S}$ internally runs $\mathcal{A}$, forwarding messages to/from the environment $\mathcal{Z}$. The simulator simulates the following interactions with $\mathcal{A}$:

- Upon receiving $(\text{KEYGENNOTIFY}, \text{sid}, P_i)$ from the ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$ about an honest generator P_i , the simulator $\mathcal{S}$ controls P_i to do the following:
 - Compute $c_i = \text{Com}(g^0)$, where $\text{Com}(g^0) = \text{hash}(g^0; r_i)$. Send $(\text{WRITE}, \text{sid}, c_i)$ to $\mathcal{G}_{\text{blockchain}}$. Wait until all parties have broadcast c_i .
 - For $P_k \in \mathcal{P}_h$, set $\sigma_{i,k} = 0$; for $P_k \in \mathcal{P}_c$, select $r_k \leftarrow \mathbb{Z}_q$ and set $\sigma_{i,k} = r_k$. Use the k 's ephemeral public key to encrypt it with hybrid encryption scheme, setting $\text{ct}_{i,k} = \text{Enc}_{\text{epk}_k}(\sigma_{i,k})$;
 - Send $(\text{WRITE}, \text{sid}, (\text{ct}_{i,1}, \dots, \text{ct}_{i,n}))$ to $\mathcal{G}_{\text{blockchain}}$.
- Once the simulated $\mathcal{G}_{\text{blockchain}}$ receives $(\text{WRITE}, \text{sid}, (\text{ct}_{i,1}, \dots, \text{ct}_{i,n}))$ from all corrupted parties, the simulator $\mathcal{S}$ does the following:
 - Use honest parties' secret keys to decrypt the corresponding ciphertexts to get $\sigma_{i,j}$, $P_i \in \mathcal{P}_c, P_j \in \mathcal{P}_h$.
 - For all $P_j \in \mathcal{P}_c$, compute $\text{psk}_j = \sum_{i \in [n]} \sigma_{i,j}$ and send $(\text{CORRUPTSHARES}, \text{sid}, (\{j, \text{psk}_j\}_{P_j \in \mathcal{P}_c}))$ to the ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$.
- Once the simulated $\mathcal{G}_{\text{blockchain}}$ receives $(\text{WRITE}, \text{sid}, (A_{j,1}, \dots, A_{j,t-1}))$ from all corrupted parties, the simulator $\mathcal{S}$ does the following:
 - If $c_i = \text{hash}(g^{u_i}; r_i)$ and $g^{\sigma_{i,k}} = \prod_{j=0}^{t-1} (A_{i,j})^{k^j}$ hold for all $P_i \in \mathcal{P}_c$, set $b = 0$, else set $b = 1$. Send $(\text{ABORT}, \text{sid}, b)$ to the ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$.
- Upon receiving $(\text{KEYGEN}, \text{sid}, (\text{psk}_i, \{\text{ppk}_j\}_{j \in [n]}))$ from the ideal functionality $\mathcal{F}_{\text{DKG}}^{t,n}[\mathbb{G}]$, the simulator $\mathcal{S}$ does the following:
 - Compute $\{A_{i,j}\}_{P_i \in \mathcal{P}_h, 0 \leq j \leq t-1}$ that satisfies $g^{\sigma_{i,k}} = \prod_{j=0}^{t-1} (A_{i,j})^{k^j}$, $P_i \in \mathcal{P}_h, P_k \in \mathcal{P}_c$ and $\text{ppk}_i = \prod_{j \in [n]} (\prod_{k=0}^{t-1} (A_{j,k})^{i^k})$, $P_i \in \mathcal{P}_h$.
 - For all honest parties $P_i \in \mathcal{P}_h$, send $(\text{WRITE}, \text{sid}, ((A_{i,0}, r_i), A_{i,1}, \dots, A_{i,t-1}))$ to $\mathcal{G}_{\text{blockchain}}$, where simulated $\mathcal{F}_{\text{RO}}$ opens c_i to $(A_{i,0}, r_i)$.
- Once an honest party receives $(\text{READPK}, \text{sid})$ from the environment $\mathcal{Z}$, compute $\text{gpk} = \prod_{i \in [n]} \text{ppk}_i$, return $(\text{READPK}, \text{sid}, \text{gpk})$.

Indistinguishability.

The indistinguishability is proven through a series of hybrid worlds $\mathcal{H}_0, \dots, \mathcal{H}_3$.

Hybrid $\mathcal{H}_0$: It is the real world execution $\text{EXEC}_{\Pi_{\text{DKG}}^{t,n}[\mathbb{G}], \mathcal{A}, \mathcal{Z}}$.

Hybrid $\mathcal{H}_1$: $\mathcal{H}_1$ is the same as $\mathcal{H}_0$ except that in $\mathcal{H}_1$, the messages $(\text{ct}_{i,1}, \dots, \text{ct}_{i,n})$ sent by the honest user P_i are replaced by simulated encryptions of 0.

Claim: If the hybrid encryption scheme HEnc is IND-CPA secure with adversary advantage $\text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$, $\mathcal{H}_0$ and

$\mathcal{H}_1$ are indistinguishable with distinguishing advantage at most $n^2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$.

Proof: The indistinguishability under chosen plaintext attack means that the adversary cannot distinguish encryption of 0 and encryption of a number. There are at most n^2 simulated ciphertexts, so the advantage in $\mathcal{H}_1$ is no more than $n^2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$.

Hybrid $\mathcal{H}_2$: $\mathcal{H}_2$ is the same as $\mathcal{H}_1$ except that in $\mathcal{H}_2$, when $P_j \in \mathcal{P}_c$ produce a valid complaint against an honest party, $\mathcal{S}$ will abort.

Claim: If the NIZK protocol is statistically sound with soundness error $\text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$, $\mathcal{H}_1$ and $\mathcal{H}_2$ are indistinguishable with distinguishing advantage at most $n(t-1) \cdot \text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$.

Proof: By the soundness of the NIZK protocol the probability that $\mathcal{S}$ will give up is negligible. There are at most $n(t-1)$ statements, so the advantage in $\mathcal{H}_1$ is no more than $n(t-1) \cdot \text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$.

Hybrid $\mathcal{H}_3$: $\mathcal{H}_3$ is the same as $\mathcal{H}_2$ except that in $\mathcal{H}_3$, c_i is opened with the simulated $\mathcal{F}_{\text{RO}}$.

Claim: $\mathcal{H}_2$ and $\mathcal{H}_3$ are perfectly indistinguishable.

Proof: As long as the encoding of $\mathcal{F}_{\text{RO}}$ has not been exhausted, the simulated $\mathcal{F}_{\text{RO}}$ is the same as real $\mathcal{F}_{\text{RO}}$.

The adversary's view of $\mathcal{H}_3$ is identical to the simulated view $\text{EXEC}_{\mathcal{F}_{\text{DKG}}^{\text{t},n}[\mathbb{G}], \mathcal{S}, \mathcal{Z}}$. Therefore, no PPT $\mathcal{Z}$ can distinguish the view of the ideal execution from the view of real execution with advantage more than $n^2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) + n(t-1) \cdot \text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$. $\square$

Theorem 4. Let the hybrid encryption scheme HEnc be IND-CPA secure with adversary advantage $\text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$. Let the NIZK protocol for decryption correctness Π_{NIZKDec} be statistically sound with soundness error $\text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$ and perfect zero-knowledge. The protocol $\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ described in fig. 10 UC-realized $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ against static corruption up to $t_1 - 1$ previous-epoch parties and up to $t_2 - 1$ next-epoch parties in the $\mathcal{G}_{\text{blockchain}}$ -hybrid model with distinguishing advantage

$$c_1 c_2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) + c_1(t_2 - 1) \cdot \text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda).$$

Proof. We first construct a simulator $\mathcal{S}$ such that any non-uniform PPT environment $\mathcal{Z}$ cannot distinguish between the ideal world execution $\text{EXEC}_{\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}], \mathcal{S}, \mathcal{Z}}$ and the real world execution $\text{EXEC}_{\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}], \mathcal{A}, \mathcal{Z}}$. In the ideal world, the parties interact with functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ and the corrupted parties are controlled by $\mathcal{S}$; in the real world, the parties $\mathcal{P} = \{P_1, \dots, P_{c_1}\}$ and $\mathcal{Q} = \{Q_1, \dots, Q_{c_2}\}$ run protocol $\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ in the $\mathcal{G}_{\text{blockchain}}$ -hybrid world and the corrupted parties are controlled by a dummy adversary $\mathcal{A}$ who simply forwards messages from/to $\mathcal{S}$.

Simulator. The simulator $\mathcal{S}$ internally runs $\mathcal{A}$, forwarding messages to/from the environment $\mathcal{Z}$. The simulator simulates the following interactions with $\mathcal{A}$:

- Upon receiving $(\text{HANDOVERNOTIFY}, \text{sid}, P_i)$ from the ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$ about an honest generator P_i , the simulator $\mathcal{S}$ controls P_i to do the following:

- Send $(\text{WRITE}, \text{sid}, \{\text{ppk}_j^{(i)}\}_{j \in [c_1]})$ to $\mathcal{G}_{\text{blockchain}}$;
- Send $(\text{READ}, \text{sid})$ to $\mathcal{G}_{\text{blockchain}}$ and set $\{\text{ppk}_j\}_{j \in [c_1]}$ as the majority of $\{\{\text{ppk}_j^{(k)}\}_{j \in [c_1]}\}_{k \in [c_1]}$;
- For $P_k \in \mathcal{P}_h$, set $\sigma_{i,k} = 0$; for $P_k \in \mathcal{P}_c$, select $r_k \leftarrow \mathbb{Z}_q$ and set $\sigma_{i,k} = r_k$. Use the k 's ephemeral public key to encrypt it with hybrid encryption scheme, setting $\text{ct}_{i,k} = \text{Enc}_{\text{epk}_k}(\sigma_{i,k})$;
- Send $(\text{WRITE}, \text{sid}, (\text{ct}_{i,1}, \dots, \text{ct}_{i,c_2}))$ to $\mathcal{G}_{\text{blockchain}}$.
- Once the simulated $\mathcal{G}_{\text{blockchain}}$ receives $(\text{WRITE}, \text{sid}, (\text{ct}_{i,1}, \dots, \text{ct}_{i,c_2}))$ from all corrupted parties, the simulator $\mathcal{S}$ does the following:
 - Use honest parties' secret keys to decrypt the corresponding ciphertexts to get $\sigma_{i,j}, P_i \in \mathcal{P}_c, Q_j \in \mathcal{Q}_h$.
 - Let the indices of the first t_1 valid messages be set $\mathcal{I}$, $\mathcal{I} \subseteq [c_1]$ and $|\mathcal{I}| = t_1$. Compute corrupted parties' new partial secret key as $\text{npsk}_i = \sum_{j \in \mathcal{I}} \lambda_j \cdot \sigma_{j,i}$, where $\lambda_j := \prod_{\ell \in \mathcal{I} \setminus \{j\}} \frac{\ell}{\ell - j}, Q_i \in \mathcal{Q}_c$;
 - Send $(\text{CORRUPTSHARES}, \text{sid}, \{j, \text{npsk}_j\}_{Q_j \in \mathcal{Q}_c})$ to the ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$.
- Once the simulated $\mathcal{G}_{\text{blockchain}}$ receives $(\text{WRITE}, \text{sid}, ((g^{u_i}, r_i), A_{j,1}, \dots, A_{j,t-1}))$ from all corrupted parties, the simulator $\mathcal{S}$ does the following:
 - If $g^{\sigma_{i,k}} = \prod_{n=0}^{t_2-1} (A_{i,n})^{k^n}$ holds for all $P_i \in \mathcal{P}_c \cap \mathcal{I}$, set $b = 0$, else set $b = 1$. Send $(\text{ABORT}, \text{sid}, b)$ to the ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$.
- Upon receiving $(\text{HANDOVER}, \text{sid}, \{\text{nppk}_j\}_{j \in [c_2]})$ from the ideal functionality $\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}]$, the simulator $\mathcal{S}$ does the following:
 - Compute $\{A_{i,j}\}_{P_i \in \mathcal{P}_h, 1 \leq j \leq t_2-1}$ satisfies $g^{\sigma_{i,k}} = \prod_{j=0}^{t_2-1} (A_{i,j})^{k^j}, P_i \in \mathcal{P}_h, Q_k \in \mathcal{Q}_c$ and $\text{nppk}_k = \prod_{j \in \mathcal{I}} (g^{\sigma_{j,k}})^{\lambda_j} = \prod_{j \in \mathcal{I}} (\prod_{n=0}^{t_2-1} (A_{j,n})^{k^n})^{\lambda_j}, Q_k \in \mathcal{Q}_h$.
 - For all honest parties $P_i \in \mathcal{P}_h$, send $(\text{WRITE}, \text{sid}, (A_{j,1}, \dots, A_{j,t-1}))$ to $\mathcal{G}_{\text{blockchain}}$.

Indistinguishability.

The indistinguishability is proven through a series of hybrid worlds $\mathcal{H}_0, \dots, \mathcal{H}_2$.

Hybrid $\mathcal{H}_0$: It is the real world protocol execution $\text{EXEC}_{\Pi_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}], \mathcal{A}, \mathcal{Z}}$.

Hybrid $\mathcal{H}_1$: $\mathcal{H}_1$ is the same as $\mathcal{H}_0$ except that in $\mathcal{H}_1$, the messages $(\text{ct}_{j,1}, \dots, \text{ct}_{j,c_2})$ sent by the honest user P_i are replaced by simulated encryptions of 0.

Claim: If the hybrid encryption scheme HEnc is IND-CPA secure with adversary advantage $\text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$, $\mathcal{H}_0$ and $\mathcal{H}_1$ are indistinguishable with distinguishing advantage at most $c_1 c_2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$.

Proof: The indistinguishability under chosen plaintext attack means that the adversary cannot distinguish encryption of 0 and encryption of a number. There are at most $c_1 c_2$ simulated ciphertexts, so the advantage in $\mathcal{H}_1$ is no more than $c_1 c_2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda)$.

Hybrid $\mathcal{H}_2$: $\mathcal{H}_2$ is the same as $\mathcal{H}_1$ except that in $\mathcal{H}_2$, when $P_j \in \mathcal{P}_c$ produce a valid complaint against an honest party, $\mathcal{S}$ will abort.

Claim: If the NIZK protocol is statistically sound with soundness error $\text{Adv}_{\text{NIZKDec}}^{\text{sound}}(\mathcal{A}, \lambda)$, $\mathcal{H}_1$ and $\mathcal{H}_2$ are indistin-

guishable with distinguishing advantage at most $c_1(t_2 - 1) \cdot \text{Adv}_{\text{NIZK}_{\text{Dec}}}^{\text{sound}}(\mathcal{A}, \lambda)$.

Proof: By the soundness of the NIZK protocol the probability that $\mathcal{S}$ will give up is negligible. There are at most $c_1(t_2 - 1)$ statements, so the advantage in $\mathcal{H}_1$ is no more than $c_1(t_2 - 1) \cdot \text{Adv}_{\text{NIZK}_{\text{Dec}}}^{\text{sound}}(\mathcal{A}, \lambda)$.

The adversary's view of $\mathcal{H}_2$ is identical to the simulated view $\text{EXEC}_{\mathcal{F}_{\text{handover}}^{t_1, t_2, c_1, c_2}[\mathbb{G}], \mathcal{S}, \mathcal{Z}}$. Therefore, no PPT $\mathcal{Z}$ can distinguish the view of the ideal execution from the view of real execution with advantage more than $c_1 c_2 \cdot \text{Adv}_{\text{HEnc}}^{\text{IND-CPA}}(\mathcal{A}, \lambda) + c_1(t_2 - 1) \cdot \text{Adv}_{\text{NIZK}_{\text{Dec}}}^{\text{sound}}(\mathcal{A}, \lambda)$. $\square$

G. Data Availability Layer

The Data Availability Layer (DA) plays a pivotal role within the DeepSafe architecture, ensuring that all transaction data can be promptly accessed, verified, and shared by all participating verification nodes. Data availability is foundational for maintaining the consistency and security of a distributed system; if transaction data is unavailable or maliciously withheld, other nodes would be unable to verify it, potentially leading to consensus failures and systemic security vulnerabilities.

In DeepSafe, the DA layer is responsible for managing and broadcasting all transaction and block data. Upon completion of verification tasks by DeepSafe nodes, the corresponding transaction data is submitted to the DA layer, guaranteeing that all nodes across the network—as well as external chains—receive consistent and complete data in a timely manner. This architecture not only ensures transparency during the verification process but also provides a foundational security guarantee for cross-chain interoperability and multi-chain collaboration, thereby reinforcing shared security across the ecosystem.

To guarantee immutability and efficient validation, the DA layer of DeepSafe incorporates cryptographic constructs such as Merkle trees and hash functions to generate verifiable commitments. Nodes can simply compare these commitments to efficiently validate whether the received data matches the originally submitted data, significantly reducing validation overhead and enhancing data transmission security.

In summary, the Data Availability Layer is an indispensable component of the overall DeepSafe architecture. It provides a trusted and verifiable data foundation for the entire ecosystem and acts as a critical pillar for shared security in a modular, multi-chain environment.

H. Governance Structure

The governance structure of DeepSafe adopts a modular design, dividing governance responsibilities across different applications and functional layers to ensure independence and non-interference between various domains. Inspired by the governance model of Polkadot, DeepSafe employs a “Nominated Proof-of-Stake (NPoS)” mechanism, relying on DeepSafe’s native token and community-driven voting power to maintain an open and decentralized governance system.

1) *Nominated Proof-of-Stake:* DeepSafe utilizes NPoS as its consensus mechanism, allowing token holders to vote and nominate validators. These validators are responsible for block production and maintaining the security of the network. This approach not only enhances network security and censorship resistance but also allows broader community participation in critical decision-making processes.

2) *Governance Participation and Voting Rights:* DeepSafe token holders play a vital role in governance by voting on proposals related to protocol upgrades, parameter adjustments, and new feature integrations. Voting power is weighted according to token holdings, ensuring that larger stakeholders have proportionally greater influence.

3) *Proposal and Voting Mechanism:* All token holders are permitted to submit governance proposals. These proposals can address network-level upgrades, parameter changes, or the addition of new modules. Each proposal undergoes a voting process and is implemented only if it receives majority approval.

4) *Council Mechanism:* DeepSafe establishes a council composed of representatives elected by the community. The council plays a pivotal role in governance, with the authority to support or reject proposals, thus safeguarding decision efficiency and preventing the passage of extreme or irrational proposals.

5) *Public Participation and Transparency:* DeepSafe emphasizes openness and transparency in governance. All proposals and voting records are publicly accessible, allowing community members to submit suggestions, participate in discussions, and monitor governance outcomes at any time.

In summary, DeepSafe’s governance model is fundamentally open and community-driven, aiming to maintain long-term decentralization, adaptability, and continuous evolution of the network.

III. KEY FEATURES

A. Security and Decentralization Guarantees

Security and decentralization are the core pillars of the Web3 ecosystem, and they form the foundational design objectives of DeepSafe. These elements are not only essential for establishing user trust but also directly influence digital asset security, censorship resistance, privacy protection, smart contract reliability, and the overall stability of blockchain networks. Thus, ensuring the security and decentralization of DeepSafe is fundamental for its large-scale adoption and continuous evolution.

DeepSafe’s pursuit of security transcends purely technical means; it embodies a broader social philosophy that reflects a collective commitment to power decentralization, self-governance, and fair order. It aims to build an equitable, transparent, and trustworthy decentralized verification infrastructure.

As a universal, cross-chain verification layer designed for multi-chain ecosystems, DeepSafe focuses on securing asset transfers, transaction signatures, state transitions, and custody operations. While mainstream blockchains like Bitcoin and Ethereum inherently possess high security and decentralization, the key infrastructures within their ecosystems — such as

cross-chain bridges, centralized exchanges, custodial services, and oracles — often remain centralized and thus prone to security breaches.

The fundamental cause of these issues lies in the absence of a truly decentralized and secure verification base layer. DeepSafe innovatively integrates Multi-party computation (MPC), zero-knowledge proofs (ZKP), and trusted execution environments (TEE) to build a trustless signature verification system. By replacing traditional centralized verification logic with mathematical and cryptographic guarantees, DeepSafe fundamentally reshapes the trust foundation of Web3.

Without altering existing chain consensus mechanisms, DeepSafe can seamlessly integrate with networks such as Bitcoin and Ethereum, providing decentralized security verification services. Its architecture is inherently secure, scalable, and verifiable, serving as the foundational security layer for the Web3 ecosystem and realizing the vision of “user self-custody, system self-verification.”

In its architectural design, DeepSafe treats both performance and security as equally critical objectives. Traditionally, decentralized verification networks face the dilemma of sacrificing performance for security or vice versa. DeepSafe, through meticulously designed multi-layered technical architecture, uniquely achieves high security while maintaining superior execution efficiency.

B. Multi-layer Hybrid Verification Architecture

The core verification process of DeepSafe is built upon a multi-layer hybrid architecture, which integrates advanced cryptographic primitives and hardware security techniques. It primarily consists of four fundamental components:

- **Ring VRF-based Random Verifier Selection:** A verifiable random function (Ring VRF) is employed to dynamically select verifiers from hundreds of globally distributed nodes. This design ensures each round of verification is unpredictable and resistant to premeditated control, thus substantially enhancing security robustness.
- **Zero-Knowledge Proofs (ZKP)** Each verifier must generate a zero-knowledge proof to demonstrate the correctness and legitimacy of its signing activity without revealing identity or signature content. This guarantees both integrity and privacy of the verification process.
- **Trusted Execution Environment (TEE)** All verification tasks are securely executed within an isolated TEE, providing hardware-level protection against external surveillance and internal collusion attacks, thereby ensuring computational confidentiality and integrity.
- **Multi-party Computation (MPC)** By adopting a threshold signature scheme, DeepSafe allows verification to be completed once a minimum number of signatures are collected, even if some nodes refuse or are offline, ensuring system availability and robustness.

This multi-layer hybrid verification architecture implements a depth-oriented defense mechanism. Through independent security guarantees at each layer, it significantly strengthens overall system redundancy and fault tolerance.

C. Performance Optimization and Scalability

In contrast to traditional multi-signature schemes requiring linear collection of signatures, DeepSafe combines Ring VRF and MPC, enabling fast verification upon reaching the predefined signature threshold. This design offers the following key advantages:

- Verification latency can be controlled at the second level, satisfying high-frequency scenarios such as cross-chain instant asset transfers and on-chain payments.
- The architecture supports horizontal scalability; the number of nodes can be dynamically adjusted based on geographic distribution and application demand without altering the core protocol.
- Signature thresholds can be flexibly configured based on risk levels, achieving an optimal balance between security and performance.

Furthermore, DeepSafe nodes provide a unified external API, which reduces integration costs and technical complexity, thereby accelerating adoption across diverse application scenarios.

D. Anti-Collusion and Censorship Resistance

By integrating random selection and identity anonymization, DeepSafe minimizes the probability of verifier collusion. Verifiers are unaware of their selection prior to signing, and all tasks are cryptographically generated and executed within TEE, preventing nodes from accessing transaction details. Moreover, node identities are encapsulated within zero-knowledge proofs, rendering it impossible to identify co-verifiers within the same round.

In extreme conditions (e.g., large-scale node downtime or concentrated adversarial node control), DeepSafe’s built-in fault tolerance and economic penalty mechanisms (e.g., staking slashing, automatic node rotation) maintain continuous network security. Emergency exit or “escape hatch” mechanisms can be activated if necessary to protect user assets.

E. High Economic Security

DeepSafe enforces a staking requirement for all participating nodes, wherein high-value tokens must be locked as collateral. Any refusal to sign, malicious activity, or prolonged downtime results in slashing of the staked assets. This economic game-theoretic design dramatically enhances honest behavior and incentivizes proactive participation, creating a security model where “the cost of attack greatly exceeds potential gains.” Consequently, the network evolves towards long-term stability and forms a self-reinforcing secure ecosystem.

IV. APPLICATION SCENARIOS

A. Cross-chain Bridges and Asset Interoperability

Cross-chain bridges have long been identified as one of the most vulnerable and high-risk infrastructures within the crypto ecosystem. Since 2021, security incidents involving cross-chain bridges have resulted in cumulative losses exceeding USD 2.5 billion. Typical cases such as Ronin Bridge, Harmony

Horizon, and Multichain illustrate that attackers often exploit single points of trust inherent in multi-signature mechanisms or compromise committee member keys to orchestrate large-scale asset theft.

The fundamental issue lies in the architectural reliance of most bridges on centralized or semi-centralized multi-signature committees. While multi-signature setups provide a security improvement over single custodial schemes, they are inherently predicated on trusting a small group of nodes or organizations. Once a subset of signatories is compromised—through bribery, coercion, or collusion—the entire bridge’s asset pool becomes vulnerable to immediate exfiltration, leaving users with no effective security recourse.

DeepSafe fundamentally redefines the trust model of cross-chain bridges through several critical innovations:

- All signing requests on bridges are executed by verification committees randomly selected via the Ring VRF algorithm within the DeepSafe network. These committee members have no fixed identities, precluding targeted attacks or premeditated collusion.
- Verification tasks are securely executed within a Trusted Execution Environment (TEE). Verifiers remain unaware of specific task contents or co-committee members’ identities, substantially mitigating risks of internal collusion or data leakage.
- Verification results are encapsulated using Zero-Knowledge Proofs (ZKP) and submitted on-chain. Only upon successful multiparty computation (MPC) threshold verification are assets minted or unlocked on the destination chain, entirely eliminating dependence on centralized trust assumptions.

By transforming asset verification from “human trust” to “cryptographic trust,” DeepSafe dramatically reduces the attack surface and enhances security, bringing cross-chain transaction security to parity with intra-chain transfers.

This architecture, rooted in verifiable randomness and hardware-enforced isolation, provides foundational security guarantees for global asset mobility and fosters true interoperability and seamless value transfer across blockchain networks.

B. Redefining Exchange Asset Security and Custodial Paradigms

Centralized exchanges, owing to their high asset concentration, have long been considered one of the most significant sources of systemic risk within the crypto domain. Historical collapses such as FTX, Mt.Gox, QuadrigaCX, and Bitfinex illustrate that when asset control resides with a small group of executives or a fixed multi-signature committee, the potential for catastrophic failure remains omnipresent. The current operational paradigm entails:

- Exchange assets are centrally managed by internal wallets (cold or hot), obliging users to place unconditional trust in the platform and its operators.
- Even in multi-signature setups, signatories are typically fixed and publicly known, rendering them susceptible to bribery, coercion, or internal collusion.

DeepSafe introduces a transformative custodial model by substituting traditional trust assumptions with a decentralized cryptographic verification layer. The core innovations include:

- The complete migration of signature authority for core exchange wallets (both cold and hot) to the DeepSafe verification network, whereby asset movement is governed by randomly selected verification committees.
- Each withdrawal or asset transfer request must be co-signed by a DeepSafe random verification committee using Multiparty Computation (MPC) and Trusted Execution Environment (TEE) technologies. Committee members operate anonymously and under strict behavioral constraints.
- Even if exchange executives maintain control over all internal IT systems and operational infrastructures, they are inherently incapable of unilaterally moving user assets, as the “signature authority” is irrevocably separated from their domain.

This trustless custodial architecture significantly reduces risks of internal malfeasance or external compromise, while providing cryptographically verifiable and publicly auditable security guarantees. The framework is especially advantageous for emerging Tier-2 and Tier-3 exchanges seeking to establish strong security and compliance differentiators, thereby enhancing user confidence and global market competitiveness.

C. AI Data Provenance and Secure Invocation

In the foreseeable future, the deep convergence of AI and blockchain technologies is anticipated to become an irreversible trajectory. The training data of AI models, inference outcomes, and invocation histories all require rigorous cryptographic verification and provenance auditing to mitigate risks such as data contamination, model misuse, and the untraceability of generative AI outputs.

Within this context, DeepSafe serves as a foundational decentralized verification layer, enabling cryptographically provable and process-verifiable data assurance for AI systems. Its core attributes include:

- Zero-Knowledge Proof (ZKP)-based verification of training data and invocation processes, ensuring compliance of data sources and authenticity of execution, without exposing any sensitive information.
- Execution of all signatures and verification tasks within Trusted Execution Environments (TEE), thereby safeguarding model parameters, user privacy, and invocation intents from unauthorized access or leaks.
- Both requesters and model service providers must undergo verification through the DeepSafe network, effectively preventing model forgery and data impersonation.

For instance, when an AI agent engages in on-chain asset management, DeFi strategy execution, or interactions with smart contracts, all corresponding transactions and signatures are validated through the DeepSafe node network. This architecture guarantees that the “robotic wallet” remains decentralized and autonomously controlled, eliminating concerns over key custody by model developers or service operators.

By adopting this structure, DeepSafe provides AI-driven agents with a highly secure, auditable, and tamper-proof verification pathway, significantly enhancing the overall trustworthiness and security of the AI-blockchain integrated ecosystem.

D. Payment Protocols and On-chain Payment Security

In the context of payment protocols—particularly in scenarios involving off-chain to on-chain settlements such as cross-border remittances and enterprise B2B transactions—traditional frameworks typically rely on trusted intermediaries or centralized clearing institutions to ensure transaction security. This approach significantly increases intermediary costs and introduces potential risks related to operational abuse, repudiation, and censorship.

DeepSafe acts as a “decentralized clearing and arbitration layer” in this scenario, with core mechanisms including:

- All payment paths, fund locking, and fund release processes must be validated and approved by the DeepSafe verification network.
- Each transaction state is auditable on-chain, and the combination of randomly selected verification nodes and Multi-Party Computation (MPC)-based signatures substantially mitigates risks of fraud and repudiation.
- Dependence on any single payment company or centralized network provider is completely eliminated, achieving an essentially “trustless” security guarantee.

By integrating DeepSafe, payment flows attain a security standard approaching that of “zero trust,” drastically simplifying protocol architecture. Furthermore, enhanced transparency and auditability elevate user confidence globally and facilitate improved regulatory compliance and operational efficiency in cross-border payments.

E. Security Assurance for Staking & Restaking

With the emergence of protocols such as EigenLayer, users are incentivized to delegate their previously staked assets to additional protocols to maximize yield. However, this process entails complex re-staking validation and risk control mechanisms, which currently depend heavily on on-chain smart contracts and a small set of arbitration nodes. Such a design introduces critical vulnerabilities, including:

- If arbitration nodes are compromised or colluded, staked assets may be maliciously slashed or incorrectly penalized.
- Validation processes are intricate, and users lack the ability to independently verify the correctness of the underlying logic on-chain.

DeepSafe introduces a robust decentralized verification framework to address these challenges, featuring:

- Validator groups are randomly selected via Ring VRF, and all validation processes are executed within secure MPC + TEE environments, fundamentally preventing collusion or malicious behavior among nodes.
- Validation results are transparently published on-chain, enabling any user or external auditor to independently verify the correctness and fairness of restaking operations.

- All reward distributions and slashing penalties are automatically triggered by the DeepSafe network, ensuring fairness, transparency, and censorship resistance.

F. Cross-Chain DEX and Asset Mirror Authenticity

Cross-chain decentralized exchanges (DEX) represent the future of asset interoperability and value exchange protocols, requiring both high-speed asset swaps and rigorous authenticity guarantees. However, conventional DEX models often rely on standalone bridge protocols or centralized intermediaries for cross-chain asset transfers, leading to constrained liquidity, asset custody risks, and a lack of trustless verification.

DeepSafe introduces the following key contributions for this scenario:

- All cross-chain asset origins and transfer paths are comprehensively verified by the DeepSafe verification network, ensuring legal provenance and operational consistency for every asset movement.
- Internal DEX protocols no longer require the maintenance of traditional bridges, as asset mapping and synchronization across chains are directly driven by the DeepSafe verification layer. This drastically simplifies architecture and significantly enhances both user experience and liquidity efficiency.
- Provision of “Inter-chain Consistency Proof,” establishing a fully trustless value transmission framework across different chains, thus enabling truly decentralized asset interoperability.

With these mechanisms, user experience in cross-chain DEX environments can reach the smoothness and speed typically associated with centralized exchanges, while fully preserving on-chain auditability and immutability as core security guarantees.

G. Decentralized Stablecoins and On-Chain Collateralization

On-chain collateralized stablecoins—especially those backed by native assets like Bitcoin (BTC)—have consistently faced two fundamental risks:

- Centralized custody of collateral assets, preventing users from exercising true control over their holdings.
- High reliance on single or minimal adjudicating parties during issuance, liquidation, and redemption processes, introducing potential censorship or internal malicious behavior.

DeepSafe integrates its self-developed CRVA (Cryptographic Random Verification Agent) network with Bitcoin-native Timelock mechanisms and Taproot technology to propose a comprehensive decentralized self-custody solution, systematically mitigating these inherent risks.

In this architecture, users deposit BTC into a 2-of-2 multisig Taproot address jointly controlled by the user and the DeepSafe CRVA network. Here, CRVA ensures decentralized signature verification, while Taproot offers enhanced privacy and scalability, eliminating traditional on-chain exposure of multisig scripts.

During voluntary redemption, both the user and the DeepSafe node network jointly execute the signature to securely

return the BTC to the user's account. If the DeepSafe network fails to cooperate within a predefined window, the Timelock mechanism activates, allowing the user to unilaterally reclaim their BTC after the lock period concludes, thus ensuring ultimate asset sovereignty.

In the case of a liquidation trigger, the DeepSafe network autonomously initiates liquidation per on-chain contract logic, transferring BTC to a unidirectional custody channel controlled by CRVA. If the user refuses cooperation, the Timelock guarantees that the asset can still be securely managed by the network, preventing malicious obstruction.

For extreme scenarios (e.g., widespread node failures or attacks), a DAO-controlled multisig fallback mechanism is designed, governed by a council comprising security agencies, community representatives, and major ecosystem participants, thereby further enhancing resilience against censorship and single points of failure.

This design not only provides logically decentralized and tamper-proof verification, but also deeply integrates Bitcoin's native security primitives (Timelock and Taproot multisig), creating a highly secure, transparent, and censorship-resistant framework for collateral management and stablecoin issuance.

Within the DeepSafe architecture, all future BTC-backed or native-asset-backed stablecoins can fully implement collateralization, minting, liquidation, and redemption operations without relying on any centralized custodial entity. This significantly elevates protocol security, system transparency, and the end-user's sovereignty over their assets.

V. CONCLUSION

DeepSafe is committed to constructing an open, decentralized, and secure cross-chain verification network, systematically addressing the critical deficiencies in "signature verification security" and "decentralized asset custody" within existing blockchain infrastructures. By integrating cutting-edge cryptographic and hardware security technologies—such as Multi-Party Computation (MPC), Zero-Knowledge Proofs (ZKP), and Trusted Execution Environments (TEE)—DeepSafe transforms traditionally trust-dependent verification processes into mathematically provable and cryptographically guaranteed trustless systems. This framework empowers core modules, including cross-chain bridges, oracles, DeFi protocols, stablecoins, and self-custodial wallets, by providing modular, composable, and verifiable security guarantees.

Without altering the consensus rules or ledger structures of Bitcoin and other primary chains, DeepSafe can seamlessly integrate as an external, independent verification layer within existing multi-chain ecosystems. It enables "trustless by design" security protection for user assets and states, fundamentally redefining the paradigm of on-chain security while laying a robust technical foundation for Web3's future autonomy, censorship resistance, and openness.

The introduction of DeepSafe represents a paradigm shift in verification infrastructure, transferring verification authority from centralized entities to the network itself. This transition drives the blockchain ecosystem toward the original vision of

"verification over trust." In the future, every chain will require verification; secure, tamper-resistant, and collusion-resistant verification capabilities will start with DeepSafe, becoming the foundational trust anchor for global data and asset cooperation and continuously fueling the decentralized future.

REFERENCES

- [1] Barreto, P.S., Kim, H.Y., Lynn, B., Scott, M.: Efficient algorithms for pairing-based cryptosystems. In: Annual international cryptology conference. pp. 354–369. Springer (2002)
- [2] Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T., Reyzin, L.: Can a public blockchain keep a secret? In: Theory of Cryptography Conference. pp. 260–290. Springer (2020)
- [3] Bernstein, D.J., Hamburg, M., Krasnova, A., Lange, T.: Elligator: elliptic-curve points indistinguishable from uniform random strings. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. pp. 967–980 (2013)
- [4] Bernstein, D.J., Lange, T., Schwabe, P.: The security impact of a new cryptographic library. In: International Conference on Cryptology and Information Security in Latin America. pp. 159–176. Springer (2012)
- [5] Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145. IEEE (2001)
- [6] Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: Theory of Cryptography Conference. pp. 61–85. Springer (2007)
- [7] Chen, J., Micali, S.: Algorand: A secure and efficient distributed ledger. *Theoretical Computer Science* **777**, 155–183 (2019)
- [8] Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
- [9] Gennaro, R., Goldfeder, S.: One round threshold ecdsa with identifiable abort. *IACR Cryptol. ePrint Arch.* **2020**, 540 (2020)
- [10] Groth, J., Kohlweiss, M.: One-out-of-many proofs: Or how to leak a secret and spend a coin. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 253–280. Springer (2015)
- [11] Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ecdsa). *International journal of information security* **1**(1), 36–63 (2001)
- [12] Lipp, M., Gruss, D., Spreitzer, R., Maurice, C., Mangard, S.: {ARMageddon}: Cache attacks on mobile devices. In: 25th USENIX Security Symposium (USENIX Security 16). pp. 549–564 (2016)
- [13] Liu, C., Wang, X.S., Nayak, K., Huang, Y., Shi, E.: Oblivm: A programming framework for secure computation. In: 2015 IEEE Symposium on Security and Privacy. pp. 359–376. IEEE (2015)
- [14] Micali, S., Rabin, M., Vadhan, S.: Verifiable random functions. In: 40th annual symposium on foundations of computer science (cat. No. 99CB37039). pp. 120–130. IEEE (1999)
- [15] Nayak, K., Fletcher, C.W., Ren, L., Chandran, N., Lokam, S.V., Shi, E., Goyal, V.: Hop: Hardware makes obfuscation practical. In: NDSS. vol. 2017, p. 0 (2017)
- [16] Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Annual international cryptology conference. pp. 129–140. Springer (1991)
- [17] Rane, A., Lin, C., Tiwari, M.: Raccoon: Closing digital {Side-Channels} through obfuscated execution. In: 24th USENIX Security Symposium (USENIX Security 15). pp. 431–446 (2015)
- [18] Schnorr, C.P.: Efficient identification and signatures for smart cards. In: Conference on the Theory and Application of Cryptology. pp. 239–252. Springer (1989)
- [19] Tramer, F., Zhang, F., Lin, H., Hubaux, J.P., Juels, A., Shi, E.: Sealed-glass proofs: Using transparent enclaves to prove and sell knowledge. In: 2017 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 19–34. IEEE (2017)
- [20] Van Bulck, J., Minkin, M., Weisse, O., Genkin, D., Kasikci, B., Piessens, F., Silberstein, M., Wenisch, T.F., Yarom, Y., Strackx, R.: Foreshadow: Extracting the keys to the intel {SGX} kingdom with transient {Out-of-Order} execution. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 991–1008 (2018)
- [21] Xu, Y., Cui, W., Peinado, M.: Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In: 2015 IEEE Symposium on Security and Privacy. pp. 640–656. IEEE (2015)